



Hochschule für Technik,
Wirtschaft und Kultur Leipzig

Bachelorarbeit
zur Erlangung des akademischen Grades

Bachelor of Science (B.Sc.)

im Bachelorstudiengang Medieninformatik
der Fakultät Informatik und Medien

Prototypische Entwicklung eines Serious Games zur spielerischen Vermittlung des objektorientierten Paradigmas

vorgelegt von
Franz Hielscher

Leipzig, den 30. August 2026

Erstprüfer: Dr. rer. nat. Nico Graebeling

Zweitprüfer: M.Sc. Felix Steffen Stolze

Abstrakt

Diese Bachelorarbeit befasst sich mit der Konzeption und prototypischen Entwicklung des Serious Games *Deep Space Debugger* zur Vermittlung grundlegender Konzepte der objektorientierten Programmierung. Das Erlernen und Verstehen der objektorientierten Konzepte stellt für eine Vielzahl von Studierenden eine Herausforderung dar. Serious Games können den Lernprozess unterstützen und das Verständnis komplexer Inhalte fördern. Ziel dieser Arbeit ist daher die Entwicklung eines Serious Games, das grundlegende Konzepte der objektorientierten Programmierung durch spielerische Mechaniken vermittelt. Hierzu wurden zunächst die theoretischen Grundlagen und bestehenden Forschungsarbeiten analysiert. Anschließend wurden auf Basis einer Literaturrecherche und zweier Experteninterviews Anforderungen an die didaktische und technische Gestaltung des Serious Games abgeleitet. Unter Berücksichtigung des MDA-Frameworks wurden diese Anforderungen in ein Spielkonzept überführt und mit der Game Engine *Godot* als funktionsfähiger Prototyp umgesetzt. Im Rahmen der prototypischen Umsetzung konnten die identifizierten Anforderungen weitgehend berücksichtigt und grundlegende Konzepte wie Klassen und Objekte, Kapselung, Vererbung, Objektkommunikation und Polymorphie in interaktive Spielmechaniken übertragen werden. *Deep Space Debugger* besitzt damit das Potenzial, die Vermittlung grundlegender Konzepte der objektorientierten Programmierung ergänzend zu unterstützen. Aussagen über die tatsächliche Lernwirksamkeit können jedoch nicht getroffen werden, da im Rahmen dieser Arbeit keine Evaluation mit der Zielgruppe durchgeführt wurde.

Inhaltsverzeichnis

Abkürzungsverzeichnis	v
1 Einleitung	1
1.1 Motivation	1
1.2 Ziel der Arbeit	2
1.3 Aufbau der Arbeit	2
2 Fachliche Grundlagen	4
2.1 Objektorientiertes Paradigma	4
2.1.1 Definition	4
2.1.2 Historische Entwicklung	5
2.1.3 Ziele	5
2.1.4 Zentrale Konzepte	6
2.1.5 Herausforderungen beim Erlernen	7
2.2 Game-Based Learning	9
2.2.1 Definition	9
2.2.2 Merkmale eines Lernspiels	9
2.2.3 Potenziale und Herausforderungen	10
2.3 Serious Games	12
2.3.1 Definition	12
2.3.2 Historische Entwicklung	12
2.3.3 Potenziale und Grenzen	13
2.3.4 Bedeutung für die Vermittlung objektorientierter Programmierung .	13
2.4 MDA-Framework	16
2.4.1 Definition	16
2.4.2 Spiele als Artefakte	17
3 Anforderungsanalyse	18
3.1 Literaturbasierte Anforderungen	18
3.1.1 Didaktische Anforderungen	18
3.1.2 Spielmechanische Anforderungen	19
3.1.3 Technische und gestalterische Anforderungen	19

3.2	Experteninterviews	21
3.2.1	Durchführung	21
3.2.2	Ergebnisse	21
3.3	Zusammenfassung der Anforderungsanalyse	24
4	Konzeption von Deep Space Debugger	25
4.1	Lernziele	25
4.2	Spielkonzept	27
4.2.1	Spielprinzip	27
4.2.2	Spielwelt und Handlung	27
4.2.3	Spielmechaniken	28
4.2.4	Core Gameplay Loop	28
4.2.5	Spielablauf	30
4.3	Didaktische Gestaltung	31
4.3.1	Vermittlung der OOP-Konzepte	31
4.3.2	Progression des Lernprozesses	32
4.3.3	Feedback und Motivation	32
4.4	Technische Konzeption	34
4.4.1	Wahl der Entwicklungsumgebung	34
4.4.2	Softwarearchitektur	34
4.4.3	Gestaltung der Benutzeroberfläche	35
5	Implementierung	36
5.1	Entwicklungsumgebung	36
5.2	Umsetzung zentraler Spielsysteme	38
5.2.1	Spielverwaltung	38
5.2.2	Interaktionssystem	39
5.2.3	Inventar- und Objektsystem	41
5.2.4	Dialogsystem	43
5.2.5	Tutorialsystem	45
5.2.6	Terminal- und Stationssystem	46
5.3	Benutzeroberfläche	48
6	Evaluation	51
6.1	Reflexion der Entwicklung	51
6.2	Umsetzung der Anforderungen	52
6.3	Limitationen	54
7	Fazit und Ausblick	55

A Interviewleitfaden	I
A.1 Zielsetzung	I
A.2 Struktur	I
B Experteninterviews	III
B.1 Transkription des ersten Experteninterviews (Experte A)	III
B.1.1 Rolle in der Programmierlehre	III
B.1.2 Herausforderungen bei der Vermittlung	IV
B.1.3 Wichtige OOP-Konzepte	IV
B.1.4 Geeignete Lehr- und Lernmethoden	IV
B.1.5 Visualisierung und Interaktivität	V
B.1.6 Lernschwierigkeiten und typische Missverständnisse	VI
B.1.7 Potenzial von Serious Games	VI
B.1.8 Grenzen und Herausforderungen von Serious Games	VII
B.1.9 Relevante Konzepte für ein Serious Game	VII
B.1.10 Zu komplexe Inhalte	VII
B.1.11 Anforderungen an ein Serious Game	VIII
B.1.12 Zugänglichkeit	VIII
B.2 Gesprächsnotizen des zweiten Experteninterviews (Experte B)	IX
B.2.1 Rolle in der Programmierlehre	IX
B.2.2 Herausforderungen bei der Vermittlung	IX
B.2.3 Wichtige OOP-Konzepte und Schwierigkeiten	IX
B.2.4 Geeignete Lehr- und Lernmethoden	X
B.2.5 Bedeutung praktischer Übungen	X
B.2.6 Visualisierung und Interaktivität	X
B.2.7 Typische Missverständnisse	X
B.2.8 Potenzial von Serious Games	XI
B.2.9 Grenzen und Herausforderungen	XI
B.2.10 Relevante Konzepte für ein Serious Game	XI
B.2.11 Schwierig spielerisch vermittelbare Inhalte	XI
B.2.12 Anforderungen an ein Serious Game	XII
Literaturverzeichnis	XIII
Abbildungsverzeichnis	XVI
Tabellenverzeichnis	XVII
Quellcodeverzeichnis	XVIII

Abkürzungsverzeichnis

DSD: Deep Space Debugger

GBL: Game-Based Learning

HTWK: Hochschule für Technik, Wirtschaft und Kultur Leipzig

HUD: Head-up-Display

MDA: Mechanics, Dynamics and Aesthetics

OOP: Objektorientierte Programmierung

1 Einleitung

Die objektorientierte Programmierung (OOP) basiert auf dem objektorientierten Paradigma, bei dem Softwaresysteme anhand von Objekten und deren Interaktion strukturiert werden. Die objektorientierte Programmierung stellt dabei die praktische Anwendung dieses Paradigmas in der Softwareentwicklung dar. Wird im weiteren Verlauf dieser Arbeit von objektorientierter Programmierung gesprochen, sind damit auch die zugrunde liegenden Konzepte des objektorientierten Paradigmas gemeint.

Das Verständnis objektorientierter Konzepte kann insbesondere für Studierende eine Herausforderung darstellen. In der Forschung wurden daher bereits unterschiedliche Ansätze zur spielerischen Vermittlung von OOP-Konzepten untersucht.[1] Serious Games bieten die Möglichkeit, fachliche Inhalte mit spielerischen Aktivitäten zu verbinden. Die vorliegende Arbeit greift diesen Ansatz auf und beschäftigt sich mit der Konzeption und prototypischen Entwicklung eines Serious Games zur Vermittlung grundlegender Konzepte der objektorientierten Programmierung.

1.1 Motivation

Bei der Vermittlung objektorientierter Programmierung müssen abstrakte Konzepte wie Klassen und Objekte, Kapselung, Vererbung oder Polymorphie verstanden und miteinander in Beziehung gesetzt werden. Schwierigkeiten beim Erlernen objektorientierter Konzepte werden auch in bestehenden Forschungsarbeiten beschrieben.[1]

Serious Games bieten die Möglichkeit, Lerninhalte mit interaktiven Spielhandlungen zu verbinden. Für die Vermittlung objektorientierter Programmierung wurden bereits verschiedene Serious Games entwickelt, die OOP-Konzepte durch unterschiedliche Ansätze aufgreifen.[1] Daraus ergibt sich die Möglichkeit, abstrakte OOP-Konzepte nicht ausschließlich textuell oder anhand von Quellcode zu vermitteln, stattdessen diese mit konkreten Objekten, Handlungen und Interaktionen innerhalb einer Spielwelt zu verknüpfen. Im Mittelpunkt dieser Arbeit steht daher nicht die Vermittlung einer bestimmten Programmiersprache, sondern das Verständnis grundlegender Konzepte des objektorientierten Paradigmas. Hierzu wird mit *Deep Space Debugger* ein sprachenunabhängiger Ansatz

verfolgt, bei dem die ausgewählten OOP-Konzepte durch spielerische und visuelle Repräsentationen vermittelt werden sollen.

1.2 Ziel der Arbeit

Das Ziel dieser Arbeit ist die Konzeption und prototypische Entwicklung des Serious Games *Deep Space Debugger* zur Vermittlung grundlegender Konzepte der objektorientierten Programmierung. Hierfür werden zunächst die theoretischen Grundlagen sowie der aktuelle Stand der Forschung betrachtet. Darauf aufbauend werden anhand einer Literaturrecherche und Experteninterviews Anforderungen an ein geeignetes Serious Game abgeleitet. Diese bilden die Grundlage für die Entwicklung eines didaktischen und technischen Konzepts sowie die prototypische Implementierung des Spiels.

Die Arbeit untersucht dabei folgende Forschungsfrage:

Inwiefern können grundlegende Konzepte des objektorientierten Paradigmas durch ein zweidimensionales Serious Game vermittelt und verständlich dargestellt werden?

Zur Beantwortung der Forschungsfrage wird untersucht, wie sich grundlegende OOP-Konzepte in spielerische Mechaniken und interaktive Aufgaben übertragen lassen und welche Anforderungen dabei zu berücksichtigen sind. Die prototypische Umsetzung dient dazu, die entwickelten Ansätze praktisch zu realisieren und hinsichtlich der abgeleiteten Anforderungen zu betrachten. Eine empirische Überprüfung der tatsächlichen Lernwirksamkeit mit der Zielgruppe ist nicht Bestandteil dieser Arbeit.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit ist in mehrere aufeinander aufbauende Kapitel gegliedert. In Kapitel 2 werden zunächst die fachlichen Grundlagen der objektorientierten Programmierung (OOP), des Game-Based Learning (GBL), der Serious Games sowie das MDA-Framework betrachtet. Darüber hinaus werden bestehende Forschungsansätze zur spielerischen Vermittlung von Programmier- und insbesondere OOP-Konzepten hinsichtlich ihrer didaktischen Gestaltung, der vermittelten Lerninhalte und ihrer Evaluation untersucht. Diese Betrachtungen schaffen die theoretische Grundlage für die weitere Arbeit.

Darauf folgt in Kapitel 3 die Anforderungsanalyse. Auf Grundlage der zuvor betrachteten

Literatur sowie der durchgeführten Experteninterviews werden didaktische, spielmechanische, technische und gestalterische Anforderungen an das zu entwickelnde Serious Game abgeleitet.

Die daraus gewonnenen Erkenntnisse bilden die Grundlage für die Konzeption von *Deep Space Debugger* (DSD) in Kapitel 4. Kapitel 5 behandelt im Folgenden die prototypische Implementierung des entwickelten Konzepts und stellt die wesentlichen technischen Systeme und deren Zusammenspiel vor.

In Kapitel 6 werden die Entwicklung und die Erfüllung der zuvor definierten Anforderungen reflektiert. Darüber hinaus wird die Forschungsfrage unter Berücksichtigung der gewonnenen Erkenntnisse betrachtet und auf bestehende Limitationen der Arbeit eingegangen.

Abschließend fasst Kapitel 7 die wesentlichen Erkenntnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2 Fachliche Grundlagen

Dieses Kapitel erläutert die fachlichen und theoretischen Grundlagen, die für die Konzeption und Implementierung von *Deep Space Debugger* relevant sind. Zunächst werden das objektorientierte Paradigma und zentrale Konzepte der objektorientierten Programmierung sowie typische Herausforderungen beim Erlernen betrachtet. Anschließend werden mit Game-Based Learning und Serious Games die didaktischen Grundlagen des spielbasierten Lernens erläutert und bestehende Forschungsansätze eingeordnet. Abschließend wird das MDA-Framework als konzeptioneller Bezugsrahmen für die Gestaltung des Serious Games vorgestellt.

2.1 Objektorientiertes Paradigma

Das objektorientierte Paradigma zählt zu den bedeutendsten Programmierparadigmen der Softwareentwicklung. Zahlreiche moderne Programmiersprachen wie Java, C++ oder C# basieren auf dessen grundlegenden Konzepten. Darüber hinaus bilden objektorientierte Prinzipien die Grundlage für eine Vielzahl von Softwareframeworks und Technologien, darunter GUI- und Web-Frameworks, Game Engines, Simulationssysteme sowie Frameworks für mobile Anwendungen. Diese stellen eine Infrastruktur bereit, in der Softwareentwickler:innen vor allem die anwendungsspezifischen Objekte und deren Verhalten definieren, während wiederkehrende Aufgaben durch das Framework übernommen werden. Aufgrund der weiten Verbreitung objektorientierter Programmierung in Studium und Praxis bildet sie die fachliche Grundlage des im Rahmen dieser Arbeit entwickelten Serious Games.[2]

2.1.1 Definition

Die objektorientierte Programmierung ist ein Programmierparadigma, das sich auf die Modellierung realer oder abstrakter Entitäten als eigenständige Objekte konzentriert. Im Mittelpunkt stehen dabei sowohl die strukturellen Eigenschaften als auch das Verhalten

dieser Objekte. Charakteristisch für die objektorientierte Programmierung ist der konzeptzentrierte Ansatz, bei dem gemeinsame Merkmale ähnlicher Objekte abstrahiert und in Klassen zusammengefasst werden. Die objektorientierte Programmierung basiert auf den vier grundlegenden Prinzipien Abstraktion, Kapselung, Vererbung und Polymorphie, die im weiteren Verlauf dieses Kapitels erläutert werden.[2]

2.1.2 Historische Entwicklung

Als Grundlage der objektorientierten Programmierung gilt die Programmiersprache Simula. Diese wurde von Dahl und Nygaard entwickelt und war ursprünglich als Programmiersprache zur Prozessbeschreibung und Simulation gedacht. Der zentrale Gedanke bestand darin, dass nicht der Quellcode selbst die Realität modelliert, sondern die durch ihn erzeugten Objekte. Der erste Entwurf entstand bereits 1963. Nach der Fertigstellung des ersten Compilers im Jahr 1965 wurde die Sprache schließlich 1967 unter dem Namen Simula 67 veröffentlicht. Mit ihr wurden zentrale Konzepte eingeführt, die später die Grundlage der objektorientierten Programmierung bildeten, darunter Klassen, Objekte, Vererbung und dynamische Objektinstanzen.[3]

Die mit Simula 67 eingeführten Konzepte eröffneten neue Möglichkeiten zur Modellierung und Strukturierung von Softwaresystemen und prägten die weitere Entwicklung objektorientierter Programmiersprachen. Gegen Ende der 1980er Jahre gewann die objektorientierte Programmierung zunehmend an Bedeutung in der Softwareentwicklung. In dieser Zeit entstanden zahlreiche objektorientierte Programmiersprachen, welche die grundlegenden Konzepte aufgegriffen und unterschiedlich weiterentwickelt haben.[3]

2.1.3 Ziele

Die Ziele des objektorientierten Paradigmas sind insbesondere auf die Beherrschung der Komplexität von Softwaresystemen fokussiert. Durch die Zerlegung eines Gesamtsystems in eigenständige Softwarekomponenten können umfangreiche Anwendungen strukturiert entwickelt, erweitert und gewartet werden. Die Modularisierung soll dabei den kognitiven Aufwand sowie die geistige Belastung von Entwickler:innen und Wartungspersonal reduzieren.[4]

Ein weiteres wesentliches Ziel besteht in der Wiederverwendbarkeit von Softwarekomponenten. Klassen und Objekte können in Softwarebibliotheken organisiert werden und in unterschiedlichen Anwendungen erneut zum Einsatz gebracht werden. Dadurch erfolgt eine Reduktion von redundantem Programmcode und Entwicklungsaufwand. Darüber

hinaus wird die Entwicklung umfangreicher Softwareprojekte unterstützt. Durch Modularisierung, Konsistenzprüfungen von Schnittstellen und weitere Softwarewerkzeuge zur Beherrschung der physischen Größe ist eine effiziente und klare Verwaltung großer Programme möglich. Zudem wird die Zusammenarbeit mehrerer Entwickler erleichtert und die langfristige Wartbarkeit der Software verbessert.[5]

2.1.4 Zentrale Konzepte

Klassen und Objekte bilden die zentralen Bestandteile der objektorientierten Programmierung. Diese dienen der Modellierung realer oder abstrakter Entitäten innerhalb eines Softwaresystems. Während eine Klasse die gemeinsame Struktur und das Verhalten einer Gruppe gleichartiger Objekte beschreibt, stellt ein Objekt eine konkrete Instanz der zugehörigen Klasse dar. Jedes Objekt besitzt einen Zustand, der durch Attribute beschrieben wird, sowie ein Verhalten, das durch Methoden definiert wird. Eine Klasse legt entsprechend fest, welche Attribute und Methoden die daraus erzeugten Objekte besitzen und wie diese umgesetzt werden.[2] Nach Wegner werden objektorientierte Systeme wesentlich durch die Existenz von Klassen charakterisiert. Klassen befinden sich gegenüber Objekten auf einer Metaebene, da sie die Struktur, das Verhalten und die Implementierung jener Objekte beschreiben, die als Instanzen einer Klasse erzeugt werden.[3]

Ein weiteres grundlegendes Konzept ist die Abstraktion. Dabei werden Gemeinsamkeiten und Unterschiede einer Menge von Entitäten betrachtet, um deren wesentliche und für die Modellierung relevante Eigenschaften herauszuarbeiten. Unwesentliche beziehungsweise für den jeweiligen Kontext nicht relevante Eigenschaften werden hingegen ausgeblendet. Auf diese Weise können verschiedene Entitäten anhand ihrer gemeinsamen Merkmale durch eine einheitliche Repräsentation beschrieben werden.[2]

Die Kapselung beschreibt die Zusammenfassung einer Repräsentation bei gleichzeitiger Verbergung interner Details. Dabei wird zwischen der Spezifikation und der konkreten Implementierung einer Entität unterschieden. Während die Spezifikation beschreibt, was eine Entität darstellt und welches Verhalten sie bereitstellt, legt die Implementierung fest, wie dieses Verhalten realisiert wird. Durch das Verbergen der Implementierungsdetails unterstützt die Kapselung zugleich die Abstraktion.[2]

Objekte stehen innerhalb eines Softwaresystems selten isoliert, sondern sind über Beziehungen miteinander verbunden. Konkrete Beziehungen zwischen einzelnen Objekten werden als Links bezeichnet, während diese auf abstrakter Ebene durch Assoziationen beschrieben werden. Zu den Formen von Assoziationen zählen unter anderem die Aggregation und die Komposition. Eine Aggregation beschreibt eine lose Ganzes-Teil-Beziehung, bei der die Teilobjekte unabhängig vom Gesamtobjekt existieren können. Bei einer Kompo-

sition ist die Existenz der Teilobjekte hingegen direkt an das Gesamtobjekt gekoppelt.[2] Die Kommunikation zwischen Objekten erfolgt über Nachrichten beziehungsweise Methodenaufrufe. Dabei wird eine Methode eines Objekts aufgerufen und gegebenenfalls durch Parameter ergänzt. Das empfangende Objekt führt daraufhin die entsprechende Methode aus, wodurch dessen Zustand oder Verhalten geändert werden kann. Auf diese Weise können Objekte miteinander interagieren und gemeinsam Abläufe innerhalb eines objektorientierten Systems realisieren.[6]

Die Vererbung beschreibt die Beziehung und Wiederverwendung bestehender Repräsentationen zur Definition neuer Repräsentationen.[2] Dabei kann eine bestehende Definition als Grundlage für eine Klasse dienen, die diese übernimmt und anschließend erweitert oder spezialisiert. Auf diese Weise können gemeinsame Strukturen und Verhaltensweisen wiederverwendet werden, ohne diese mehrfach definieren zu müssen. Die Vererbung ermöglicht somit eine schrittweise Spezialisierung bestehender Abstraktionen und kann gleichzeitig die Wiederholung von Programmcode reduzieren.[7]

Polymorphie beschreibt die Möglichkeit, unterschiedliche Implementierungen auf Grundlage einer gemeinsamen Spezifikation bereitzustellen. Dadurch können verschiedene Objekte über dieselbe Schnittstelle angesprochen werden, obwohl sich die konkrete Umsetzung ihres Verhaltens unterscheidet.[2] Besitzen unterschiedliche Objekte eine gemeinsame Schnittstelle, kann ein Programm dieselbe Operation auf diese anwenden, ohne deren konkrete Implementierung kennen zu müssen. Die Verantwortung für die Ausführung liegt dabei beim jeweiligen Objekt, wodurch unterschiedliche Objekttypen einheitlich behandelt werden können.[7]

2.1.5 Herausforderungen beim Erlernen

Trotz ihrer weiten Verbreitung gilt die objektorientierte Programmierung als anspruchsvoll zu erlernen. Eine Ursache liegt unter anderem darin, dass objektorientierte Programmierung ein Umdenken gegenüber prozeduralen Programmieransätzen erfordert. Entwickler:innen müssen zunächst die semantische Struktur eines Problems analysieren, geeignete Objekte identifizieren und deren Beziehungen modellieren, bevor eine konkrete Implementierung erfolgen kann. Dadurch unterscheidet sich die objektorientierte Programmierung deutlich von einem rein prozeduralen Vorgehen, bei dem der Fokus stärker auf der Abfolge der einzelnen Anweisungen liegt.[4]

Eine häufige Herausforderung besteht in der Definition geeigneter Klassen auf Grundlage identifizierter Objekte. Anfänger:innen orientieren sich häufig an Vorgehensweisen, die sie in Programmierkursen erlernt haben. Dabei werden zunächst Entitäten der Anwendungsdomäne identifiziert und anschließend Klassen sowie Methoden abgeleitet. Dies kann zu

fehlerhaften Klassenmodellen führen und eine nachträgliche Überarbeitung der Softwarestruktur erforderlich machen. Eine weitere Herausforderung besteht in der Zerlegung einer Problemstellung sowie der sinnvollen Zuordnung von Funktionalität zu einzelnen Objekten und Klassen. Insbesondere Anfänger:innen haben Schwierigkeiten, deklarative und prozedurale Aspekte einer Lösung miteinander zu verbinden. Anstatt die Programmlogik auf mehrere funktionale Einheiten zu verteilen, wird sie häufig einer einzelnen Klasse zugeordnet. Ursachen hierfür sind unter anderem ein unzureichendes Verständnis der objektorientierten Konzepte sowie die Übertragung prozeduraler Denkweisen. Dies kann dazu führen, dass bereits entworfene Klassenmodelle grundlegend überarbeitet werden müssen. Ein weiterer Schwerpunkt der Lernschwierigkeiten liegt im Verständnis der Vererbung. Während Menschen natürliche Hierarchien im Alltag intuitiv anhand gemeinsamer Merkmale bilden, unterscheidet sich die objektorientierte Vererbung grundlegend von diesem natürlichen Verständnis. Dadurch entstehen fehlerhafte Vererbungsstrukturen oder ein unangemessener und zu häufiger Einsatz des Vererbungsprinzips. Besondere Schwierigkeiten bereitet die Vererbung von Funktionalitäten und Methoden, wodurch Softwarekomponenten mit Vererbungsbeziehungen eine erhöhte Fehleranfälligkeit aufweisen können. Darüber hinaus zeigen Untersuchungen, dass Entwickler:innen Klassen teilweise fälschlicherweise als bloße Menge von Objekten interpretieren. Folge dessen ist, dass dem Klassenkonzept die Eigenschaften einer Menge zugewiesen werden, was die Entwicklung geeigneter objektorientierter Modelle zusätzlich erschwert.[4]

Die beschriebenen Lernschwierigkeiten zeigen sich auch in weiteren Untersuchungen zur Vermittlung objektorientierter Programmierung. So werden insbesondere Vererbung, Abstraktion und Polymorphie als herausfordernde Konzepte für Studierende beschrieben. Gleichzeitig wird auf Probleme klassischer Vermittlungsformen sowie unterschiedliche Lernvoraussetzungen, wie geringe Programmiererfahrung, begrenzte Unterrichtszeiten und unterschiedliche Lerntypen hingewiesen. Vor diesem Hintergrund werden spielbasierte Ansätze als Möglichkeit untersucht, die Vermittlung von OOP-Konzepten ergänzend zu unterstützen.[8]

Die beschriebenen Herausforderungen verdeutlichen, dass das Erlernen der objektorientierten Programmierung sowohl fachliche als auch didaktische Anforderungen mit sich bringt. Die Erkenntnisse aus der Literatur werden im weiteren Verlauf dieser Arbeit durch die Ergebnisse der Experteninterviews ergänzt und bilden eine Grundlage für die Ableitung der Anforderungen an das Serious Game *Deep Space Debugger*.

2.2 Game-Based Learning

Game-Based Learning (GBL) beschreibt eine gezielte Einbindung von Spielen in Lehr- und Lernprozesse. Dabei werden spielerische Elemente mit fachlichen Lernzielen verbunden, um eine aktive Auseinandersetzung mit den Lerninhalten zu fördern. Da Serious Games (Kapitel 2.3) als eine konkrete Ausprägung dieses Ansatzes eingesetzt werden können, bildet Game-Based Learning eine didaktische Grundlage für die weitere Betrachtung dieser Arbeit.[9]

2.2.1 Definition

Game-Based Learning bezeichnet einen pädagogischen Lehr- und Lernansatz, bei dem Spiele gezielt zur Unterstützung von Lernprozessen eingesetzt werden, um Lernende stärker zu motivieren und aktiv einzubinden.[10] Die Lernaufgaben werden dabei so gestaltet, dass fachliche Inhalte in den Spielverlauf integriert und schrittweise vermittelt werden. Zur Umsetzung können Spielelemente wie Punktesysteme, Belohnungen, Fortschrittsmechaniken, Wettbewerbe sowie unmittelbare Feedbackmechanismen eingesetzt werden. Game-Based Learning findet heute in unterschiedlichen Bildungsbereichen Anwendung, darunter Schulen, Hochschulen sowie die berufliche Aus- und Weiterbildung.[9]

2.2.2 Merkmale eines Lernspiels

Ein zentrales Merkmal des Game-Based Learning besteht darin, dass die Lerninhalte eng mit den Spielinhalten verbunden sind. Der Wissenserwerb erfolgt somit nicht unabhängig vom Spiel, sondern durch die aktive Auseinandersetzung mit dessen Spielmechaniken. Charakteristisch sind interaktive Spielwelten mit klar definierten Regeln, Zielen, visuellen Elementen und unmittelbaren Rückmeldungen auf die Handlungen der Spielenden. Dadurch wird eine kontinuierliche Interaktion mit den Lerninhalten ermöglicht.[11]

Darüber hinaus spielen motivationale Faktoren eine wichtige Rolle. Nach Thomas W. Malone werden Videospiele insbesondere durch die Elemente Fantasie, Neugier, Herausforderung und Kontrolle geprägt.[12] Fantasie schafft einen narrativen Kontext, der das Interesse der Lernenden steigern kann. Neugier wird durch die fortlaufende Einführung neuer Informationen und nicht deterministischer Ergebnisse aufrechterhalten. Ein angemessener Schwierigkeitsgrad sorgt für kontinuierliche Herausforderungen, während die Möglichkeit, Entscheidungen zu treffen und deren Konsequenzen zu erleben, den Spielenden ein Gefühl der Kontrolle vermittelt.[11]

Ein weiteres Merkmal des Game-Based Learning ist die Verknüpfung von Spielerfahrung

und Reflexion. Wiederholte Spielzyklen und unmittelbares Feedback ermöglichen es den Lernenden, aus den Folgen ihrer Handlungen zu lernen. Eine besondere Rolle nimmt dabei der Debriefing-Prozess ein, der als Bindeglied von Spielerfahrung und Lernziel dient. Durch die Reflexion des Erlebten können die im Spiel gewonnenen Erkenntnisse auf praktische Anwendungssituationen übertragen werden.[11] Dieser Zusammenhang wird durch das Game-Based-Learning-Modell in Abbildung 2.1 veranschaulicht.

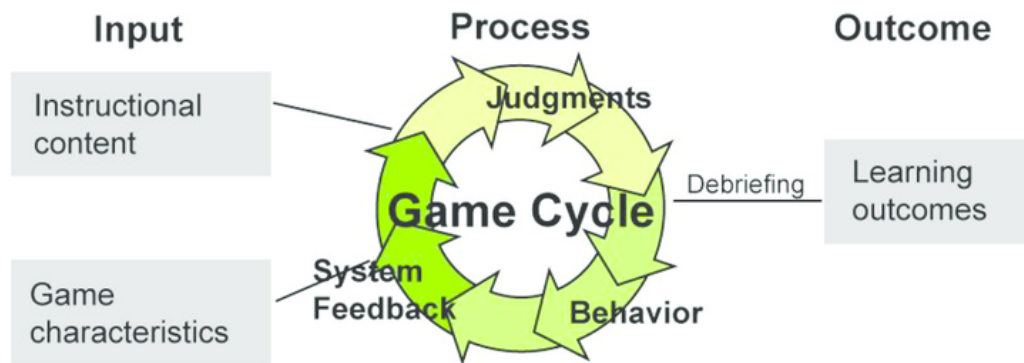


ABBILDUNG 2.1: Game-Based-Learning-Modell nach Garrison et al. [13]

Die Bedeutung der Reflexionsphase zeigt sich auch in bestehenden spielbasierten Ansätzen zur Vermittlung von Programmierkonzepten. Djelil und Sanchez heben bei der Entwicklung von *Progo* das Debriefing als wesentlichen Bestandteil des Lernprozesses hervor. Die Reflexion der Spielerfahrung soll dabei unterstützen, die im Spiel dargestellten Zusammenhänge zu erkennen und das erworbene Wissen auf andere Situationen zu übertragen. Zudem wird die Bedeutung von verständlichen Analogien zwischen Spiel und Fachinhalt betont. Dies ermöglicht den Lernenden ein zugängliches Erfahren der abstrakten Konzepte.[14]

2.2.3 Potenziale und Herausforderungen

Das Game-Based Learning bietet verschiedene Potenziale für die Gestaltung von Lernprozessen. Durch die Einbettung von Lerninhalten in einen spielerischen Kontext können die Motivation und der Lernerfolg der Lernenden positiv beeinflusst werden. Gleichzeitig ermöglicht der Ansatz eine aktive Auseinandersetzung mit den Lerninhalten, indem Spielende Entscheidungen treffen und deren Konsequenzen unmittelbar erleben. Direktes Feedback unterstützt dabei die Einschätzung des eigenen Lernfortschritts. Darüber hinaus können Fehler ohne reale negative Auswirkungen gemacht und unterschiedliche Lösungswege in einer geschützten Umgebung erprobt werden.[9]

Diese Potenziale zeigen sich auch in Untersuchungen zum Einsatz spielbasierter Ansätze in der Programmierlehre. Bei der Evaluation von *ProGames* konnte in der Experimentalgruppe eine signifikante Verbesserung der Lernergebnisse sowie des Wissenszuwachses festgestellt werden. Darüber hinaus zeigte sich, dass die Akzeptanz der spielerischen Lernumgebung höher ausfiel, wenn die Gestaltung den persönlichen Interessen der Studierenden entsprach. Dies unterstreicht neben den möglichen Einfluss auf den Lernerfolg insbesondere die Bedeutung einer zielgruppenorientierten und motivierenden Gestaltung der Lernumgebung. [15]

Trotz dieser Potenziale ist der Einsatz von Game-Based Learning mit verschiedenen Herausforderungen verbunden. Grundsätzlich muss sichergestellt werden, dass der Einsatz des Spiels tatsächlich den Lernprozess unterstützt und nicht ausschließlich aufgrund der verwendeten Technologie erfolgt. Darüber hinaus müssen die technischen Kompetenzen der Zielgruppe berücksichtigt werden, damit die Anwendung nicht selbst zu einer Lernhürde wird. Eine weitere Herausforderung besteht in der Entwicklung geeigneter Lernspiele, die fachliche Inhalte didaktisch sinnvoll vermitteln und zugleich motivierende Spieleigenschaften aufweisen. Die Verbindung dieser Anforderungen kann einen hohen Entwicklungsaufwand verursachen und erfordert eine Abwägung zwischen pädagogischem Nutzen sowie dem benötigten Zeit- und Ressourcenaufwand.[9]

2.3 Serious Games

Aufbauend auf dem zuvor beschriebenen Game-Based Learning (Kapitel 2.2) stellen Serious Games eine konkrete Möglichkeit dar, spielbasierte Lernprozesse umzusetzen.[16] Sie verbinden die Eigenschaften digitaler Spiele mit einem über den Unterhaltungszweck hinausgehenden Ziel und finden unter anderem in Bildung, Ausbildung und weiteren Ausbildungsbereichen Verwendung.[9] Im Folgenden werden ihre grundlegenden Eigenschaften sowie ihre Bedeutung für die Vermittlung objektorientierter Programmierung betrachtet.

2.3.1 Definition

Serious Games sind digitale Spiele, die neben dem Unterhaltungsaspekt ein ernstes beziehungsweise produktives Ziel verfolgen und eine Ausprägung des Game-Based Learning (siehe Kapitel 2.2) darstellen. Spielmechaniken, Storytelling und audiovisuelles Design werden dabei gezielt mit fachlichen beziehungsweise didaktischen Inhalten verbunden, um ein definiertes Bildungsziel zu verfolgen.[17]

Ein wesentliches Merkmal von Serious Games ist die Simulation realitätsnaher Situationen, wodurch erfahrungsbasiertes Lernen ermöglicht wird. Im Gegensatz zur Gamification handelt es sich bei Serious Games um eigenständige Spiele, während bei der Gamification einzelne spieltypische Elemente in einen spielfremden Kontext übertragen werden. Die Abgrenzung beider Ansätze ist in der Praxis jedoch nicht immer eindeutig.[16]

2.3.2 Historische Entwicklung

Die ersten Ansätze des Konzepts Serious Games gehen auf Clark C. Abt zurück. Er beschrieb bereits in den 1970er Jahren den Einsatz von Spielen als Instrument der Wissensvermittlung und betonte deren Potenzial zur Förderung des Lernerfolgs.[18] Der Begriff wurde später insbesondere durch Ben Sawyer geprägt. Nach dessen Verständnis handelt es sich um softwarebasierte Spiele, die mithilfe audiovisueller Simulationen sowie spieltypischer Elemente wie Wettbewerb, Interaktion und Vorstellungskraft die Vermittlung von Wissen und Kompetenzen unterstützen. Ziel ist es, definierte Lerninhalte auch ohne eine kontinuierliche Betreuung effizient und nachhaltig zu vermitteln.[19][16]

2.3.3 Potenziale und Grenzen

Serious Games bieten die Möglichkeit, fachliche Inhalte in einen spielerischen Anwendungskontext zu übertragen und dadurch einen interaktiven Zugang zu komplexen Themenbereichen zu schaffen.[17] Im Bereich der Programmierlehre können Serious Games insbesondere die Vermittlung grundlegender Konzepte unterstützen. Gleichzeitig ermöglichen die im Spiel erfassten Handlungen und Entscheidungen eine kontinuierliche Rückmeldung zum Lernfortschritt, beispielsweise durch integrierte In-Game-Assessments.[20] Die Wirksamkeit von Serious Games hängt jedoch wesentlich von ihrer didaktischen und spielerischen Gestaltung ab.[20] Insbesondere lernirrelevante Spielelemente können zusätzliche kognitive Belastungen verursachen dadurch Ressourcen beanspruchen, die für die eigentliche Informationsverarbeitung benötigt werden. Aus diesem Grund ist eine ausgewogene Verbindung von Lern- und Spielelementen entscheidend, damit motivierende Spieleigenschaften den Lernprozess unterstützen und nicht von diesem ablenken.[10]

2.3.4 Bedeutung für die Vermittlung objektorientierter Programmierung

Serious Games eignen sich grundsätzlich zur Vermittlung von Grundlagen der Programmierung und können insbesondere Anfänger:innen beim Erwerb neuer Kompetenzen unterstützen. Im Bereich der objektorientierten Programmierung existieren verschiedene Ansätze, die abstrakte Konzepte durch spielerische Handlungen, visuelle Darstellungen und interaktive Lernumgebungen erfahrbar machen. Bestehende Forschungsarbeiten unterscheiden sich dabei sowohl hinsichtlich der vermittelten OOP-Konzepte als auch ihrer didaktischen und spielmechanischen Gestaltung.[21, 14, 22, 20, 23]

Einen umfangreicheren Ansatz zur spielerischen Vermittlung objektorientierter Konzepte stellt *Odyssey of Phoenix* dar. Das mobile 2D-Rollenspiel integriert explizit objektorientierte Konzepte wie Klassen, Objekte, Attribute, Methoden, Kapselung, Vererbung und Polymorphie unmittelbar in die Spielhandlung und kombiniert diese mit Quests, NPCs, Crafting-Elementen, Kämpfen und einem Belohnungssystem. Die Konzepte werden dabei schrittweise vermittelt. Die Evaluation des Spiels deutet darauf hin, dass der spielbasierte Ansatz den Wissenserwerb unterstützen kann. Aufgrund der begrenzten Stichprobe sind die Ergebnisse jedoch nur eingeschränkt verallgemeinerbar. Der Ansatz verdeutlicht damit, dass auch mehrere aufeinander aufbauende OOP-Konzepte in eine zusammenhängende Spielhandlung integriert und schrittweise vermittelt werden können.[21]

Einen weiteren Ansatz zur Vermittlung objektorientierter Konzepte verfolgen Djelil und Sanchez mit dem Serious Game *Progo*. Hier werden abstrakte Konzepte wie Klassen und

Objekte mithilfe einer durchgängigen dreidimensionalen Konstruktionsmetapher in eine spielbare Situation überführt. Klassen werden durch Modelle und Objekte durch deren Instanzen repräsentiert, während Attribute über Eigenschaften wie Farbe, Rotation oder Position und Methoden durch ausführbare Aktionen dargestellt werden. Die Vermittlung folgt dabei einem schrittweisen Object-First-Ansatz, bei dem zunächst vorhandene Objekte verwendet werden, bevor eigene Klassen sowie komplexere Beziehungen wie Vererbung, Aggregation und Komposition behandelt werden. Der Ansatz zeigt, wie abstrakte OOP-Konzepte durch visuelle Metaphern in konkrete und interaktiv erfahrbare Zusammenhänge übertragen werden können.[14]

Weitere Forschungsarbeiten verfolgen ähnliche Ansätze. Wong und Yatim entwickelten mit *ZTech de Object-Oriented* ein 2D-Rollenspiel, das objektorientierte Konzepte mithilfe von Quests, NPCs und Minispielen vermittelt. Die Autoren betonen insbesondere die Bedeutung einer verständlichen und übersichtlichen Benutzeroberfläche, da ein kompliziertes Interface schnell zu Frustration führen und vom Lernen ablenken kann. Darüber hinaus werden eine motivierende Handlung sowie eine schrittweise Progression der Lerninhalte hervorgehoben.[22] Elaachak und Bouhorma präsentieren dagegen ein mobiles Serious Game, das die vier grundlegenden objektorientierten Konzepte mithilfe von Drag-and-Drop, Puzzle-Elementen sowie Strategieelementen vermittelt und den Lernfortschritt kontinuierlich durch ein integriertes In-Game-Assessment erfasst.[20] Beide Ansätze verdeutlichen, dass neben der eigentlichen Darstellung der OOP-Konzepte auch die Gestaltung der Benutzeroberfläche, die Progression sowie das kontinuierliche Feedback an die Lernenden eine wichtige Rolle für die spielerische Vermittlung einnehmen.

Die unterschiedlichen Ansätze zur spielerischen Vermittlung objektorientierter Programmierung werden auch in der systematischen Literaturübersicht von Abbasi et al. deutlich. Die betrachteten Arbeiten unterscheiden sich insbesondere hinsichtlich der didaktischen Herangehensweise und der Integration von OOP-Konzepten in den Lernprozess. Dabei werden unter anderem Object-First-, Game-First- und GUI-First-Ansätze eingesetzt, um den Einstieg in die objektorientierte Programmierung zu unterstützen. Die Untersuchung verdeutlicht zugleich, dass Serious Games und spielbasierte Lernumgebungen auf unterschiedliche Weise zur Vermittlung von OOP-Konzepten eingesetzt werden können.[23]

Die betrachteten Forschungsarbeiten zeigen insgesamt, dass Serious Games grundsätzlich für die Vermittlung der objektorientierten Programmierung geeignet sind und diese unterstützen können. Die bestehenden Ansätze unterscheiden sich jedoch hinsichtlich ihres Umfangs und ihrer inhaltlichen Ausrichtung. Während einige Anwendungen vorwiegend grundlegende Programmierkonzepte behandeln, konzentrieren sich andere gezielt auf ausgewählte Konzepte der objektorientierten Programmierung. Auch hinsichtlich der didaktischen und spielerischen Gestaltung zeigen sich unterschiedliche Ansätze. Wiederkehrende Elemente sind insbesondere die schrittweise Vermittlung der Lerninhalte, deren

Einbettung in konkrete Spielhandlungen, die Verwendung visueller Metaphern sowie unmittelbares Feedback an die Lernenden. Eine Verbindung dieser Ansätze mit einer zusammenhängenden Spielwelt und einer gezielten Berücksichtigung der in Abschnitt 2.1.5 beschriebenen Lernschwierigkeiten findet sich jedoch nur eingeschränkt.

An dieser Stelle setzt die vorliegende Arbeit mit *Deep Space Debugger* an. Ziel ist es, grundlegende Konzepte der objektorientierten Programmierung unabhängig von einer konkreten Programmiersprache in einer zusammenhängenden Spielwelt zu vermitteln und unmittelbar mit den Spielhandlungen zu verknüpfen. Die Gestaltung berücksichtigt dabei sowohl die Erkenntnisse der dargestellten Forschungsarbeiten als auch die im Rahmen der Anforderungsanalyse untersuchten didaktischen Anforderungen.

2.4 MDA-Framework

Das MDA-Framework (Mechanics, Dynamics, Aesthetics) ist ein formaler Ansatz zur Analyse und Gestaltung von Computerspielen. Es wurde von Robin Hunicke, Marc LeBlanc und Robert Zubek entwickelt und verfolgt das Ziel, die Lücke zwischen Game Design, Spielentwicklung, Spielkritik sowie der technischen Spielforschung zu füllen.[24, 25] Das Framework findet insbesondere in Forschung und Lehre weite Verbreitung und dient als theoretischer Bezugsrahmen für die systematische Gestaltung von Spielerlebnissen.[26]

2.4.1 Definition

Das MDA-Framework unterteilt Computerspiele in drei miteinander verbundenen Komponenten Mechanics, Dynamics, und Aesthetics. Dabei unterscheidet es zwischen der Perspektive der Entwickler:innen und der Spielenden. Entwickler:innen gestalten ausgehend von den Mechaniken die daraus entstehenden Dynamiken bis hin zu den angestrebten Ästhetiken. Spielende nehmen das Spiel in umgekehrter Reihenfolge wahr, indem sie zunächst die resultierende Spielerfahrung erleben.[24, 25]

Mechaniken (Mechanics) beschreiben die Bestandteile, Steuerelemente und Abläufe eines Computerspiels. Sie definieren die Regeln und Komponenten, die im Spiel implementiert sind. Hierzu zählen unter anderem grundlegende Aktionen, Datenstrukturen, Algorithmen, implementierte Funktionen der Game Engine sowie verschiedene Spielelemente. Die Mechaniken legen fest, welche Aktionen möglich sind und bilden somit das technologische Grundgerüst des Spiels.[24, 25]

Dynamiken (Dynamics) beschreiben das Verhalten des Spiels während der Laufzeit. Sie entstehen durch das Zusammenwirken der implementierten Mechaniken mit den Eingaben der Spielenden sowie den Wechselwirkungen verschiedener Spielsysteme.[24] Daraus ergeben sich beispielsweise unterschiedliche Strategien, Fortschritt, Wettbewerb oder andere Formen der Zusammenarbeit. [25]

Ästhetiken (Aesthetics) beschreiben die Spielerfahrung, die bei den Spielenden durch das Zusammenspiel von Mechaniken und Dynamiken hervorgerufen wird.[24] Hierzu zählen beispielsweise das Erleben von Herausforderung, Entdeckung, Fantasie, Gemeinschaft oder narrativen Elementen.[25]

2.4.2 Spiele als Artefakte

Ein grundlegender Gedanke des MDA-Frameworks besteht darin, Computerspiele als Artefakte zu verstehen. Im Gegensatz zu klassischen Medien besteht der Inhalt eines Spiels nicht ausschließlich aus den dargestellten Bildern, Klängen oder Texten. Entscheidend ist das Verhalten des Spiels, das durch die Interaktion zwischen den Spielenden und dem Spielsystem entsteht. Da jede Person unterschiedliche Entscheidungen trifft und dadurch eigene individuelle Erfahrungen macht, werden Spiele grundlegend subjektiv wahrgenommen. Das Framework ermöglicht die systematische Analyse und gezielte Gestaltung von Spielerlebnissen.[24]

Aufgrund dieser systematischen Betrachtung dient das MDA-Framework im weiteren Verlauf dieser Arbeit als konzeptioneller Bezugsrahmen für die Entwicklung und Evaluation von *Deep Space Debugger*. die Gestaltung der Spielmechaniken, der daraus entstehenden Dynamiken und der angestrebten Ästhetiken orientiert sich an den Grundprinzipien des MDA-Frameworks.

3 Anforderungsanalyse

Aufbauend auf den in Kapitel 2 dargestellten fachlichen und didaktischen Grundlagen werden im Folgenden Anforderungen an die Konzeption von *DSD* abgeleitet. Die Entwicklung eines Serious Games erfordert die Berücksichtigung unterschiedlicher fachlicher, didaktischer, spielmechanischer und technischer Anforderungen. Diese lassen sich sowohl aus wissenschaftlichen Erkenntnissen als auch aus praktischen Erfahrungen der Lehre ableiten. Das folgende Kapitel betrachtet zunächst literaturbasierte Anforderungen an die Gestaltung von Serious Games. Anschließend werden die Ergebnisse der Experteninterviews vorgestellt und ausgewertet. Die Zusammenführung beider Perspektiven bildet die Grundlage für die Konzeption (Kapitel 4) und Entwicklung (Kapitel 5) von *DSD*.

3.1 Literaturbasierte Anforderungen

Die in den Abschnitten 2.2 und 2.3 dargestellten Grundlagen verdeutlichen, dass die Wirksamkeit eines Serious Games wesentlich von der Verbindung zwischen Lerninhalten und spielerischer Gestaltung abhängt. Für die Konzeption von *DSD* (siehe Kapitel 4) wurden daher ausgewählte Forschungsarbeiten zu Serious Games, Game-Based Learning sowie bestehenden Entwicklungs- und Designansätzen hinsichtlich konkreter Gestaltungsforderungen untersucht. Die daraus abgeleiteten Anforderungen werden im Folgenden dargestellt.

3.1.1 Didaktische Anforderungen

Aus didaktischer Perspektive sollte ein Serious Game den Lernprozess gezielt unterstützen und die pädagogischen Ziele bereits während der Gestaltung berücksichtigen. Pacheco-Velazquez et al. betonen hierzu die frühzeitige Definition konkreter Lernziele, die als Grundlage für die weitere Gestaltung des Spiels dienen.[27] Die Spielmechaniken und Spielziele sollten anschließend an diesen Lernzielen ausgerichtet werden, sodass die Handlungen der Spielenden die angestrebten Kenntnisse beziehungsweise Fähigkeiten aufgreifen

und anwenden.[27]

Die Lerninhalte sollten außerdem schrittweise vermittelt werden. Sowohl Pacheco-Velazquez et al. als auch Ravyse et al. beschreiben eine strukturierte Progression, bei der Spielabschnitte beziehungsweise Herausforderungen zunehmend komplexer gestaltet sowie neue Inhalte auf vorherigen Erfahrungen aufgebaut werden können.[27, 28]

Ein weiterer wesentlicher Bestandteil ist unmittelbares Feedback. Dieses ermöglicht den Spielenden, die Folgen ihrer Handlungen direkt nachzuvollziehen und den eigenen Fortschritt einzuschätzen.[29, 28] Ravyse et al. empfehlen darüber hinaus die kontinuierliche Darstellung von Status- und Fortschrittsinformationen, während Pacheco-Velazquez et al. die Erfassung von Spielhandlungen und Lernergebnissen als Basis für Feedback und die Nachvollziehbarkeit des Lernfortschritts hervorheben.[28, 27]

3.1.2 Spielmechanische Anforderungen

Neben der didaktischen Zielsetzung muss ein Serious Game auch eine motivierende Spielerfahrung ermöglichen. Pacheco-Velazquez et al. führen hierzu insbesondere Herausforderungen, Belohnungen und eine immersive Erzählung als Spielelemente an, die die Aufmerksamkeit und das Interesse der Lernenden unterstützen können.[27] Auch Ravyse et al. identifizierten unter anderem Narration, Interaktion sowie Feedback als zentrale Erfolgsfaktoren für lernwirksame Serious Games.[28]

Die Inhalte müssen dabei möglichst eng in die Spielhandlungen eingebettet werden. Herausforderungen und Aufgaben innerhalb des Spiels sollten mit den behandelten Inhalten übereinstimmen und die Spielenden durch ihre Entscheidungen zur aktiven Anwendung des vermittelten Wissens anregen.[27] Eine zu starke Dominanz spielerischer Elemente ist hingegen zu vermeiden, da diese vom eigentlichen Lerninhalt ablenken kann.[29, 28]

Serious Games sollten darüber hinaus Möglichkeiten zur Exploration und zum experimentellen Handeln bieten. Spielende können verschiedene Handlungen ausprobieren, deren Auswirkungen beobachten und aus Fehlentscheidungen lernen.[29, 27] Dabei bietet die virtuelle Spielumgebung die Möglichkeit, Entscheidungen in einem gesicherten Kontext ohne reale Konsequenzen zu erproben.[29, 28]

3.1.3 Technische und gestalterische Anforderungen

Die technische und gestalterische Umsetzung sollte die Nutzung des Serious Games unterstützen und keine zusätzliche Lernhürde darstellen. Ravyse et al. empfehlen hierzu insbesondere eine intuitive Benutzeroberfläche, reduzierte Steuerungsmechaniken und eine übersichtliche Darstellung relevanter Informationen.[28]

Darüber hinaus sollte die Gestaltung an den Eigenschaften und Kenntnissen der Zielgruppe orientiert werden. Pacheco-Velazquez et al. heben hervor, dass bereits in der Designphase das Nutzerprofil, das Vorwissen und die Vertrautheit der Zielgruppe mit dem Lerngegenstand berücksichtigt werden sollten.[27] Auch Ravyse et al. empfehlen die frühzeitige Einbindung der vorgesehenen Zielgruppe in Design- und Testphasen.[28]

Die Qualität des Serious Games sollte während des Entwicklungsprozesses untersucht werden. Dabei sind sowohl technische Funktionalität als auch die Spielerfahrung und die pädagogische Wirksamkeit zu berücksichtigen. Pacheco-Velazquez et al. unterscheiden hierzu zwischen technischem, spielerischem und pädagogischem Testing und schlagen insbesondere die Kontrolle der Lernziele und der Motivation sowie des Engagements der Spielenden vor.[27] Auch Belloti et al. weisen auf den Bedarf umfangreicher empirischer Evaluationen hin, um die tatsächliche Lernwirksamkeit von Serious Games beurteilen zu können.[29]

3.2 Experteninterviews

Zur Ergänzung der literaturbasierten Anforderungen wurden zwei leitfadengestützte Experteninterviews durchgeführt. Ziel war es, typische Lernschwierigkeiten bei der Vermittlung objektorientierter Programmierung zu identifizieren und daraus spezifische Anforderungen an ein Serious Game abzuleiten. Die beiden befragten Personen verfügen über Lehrerfahrung im Bereich der objektorientierten Programmierung und betreuen Lehrveranstaltungen in unterschiedlichen Studienphasen. Zur anonymisierten Darstellung werden sie im Folgenden als Experte A und Experte B referenziert.

3.2.1 Durchführung

Die Experteninterviews wurden als leitfadengestützte Einzelinterviews durchgeführt. Grundlage bildete ein zuvor entwickelter Leitfaden, der sich an der Forschungsfrage dieser Arbeit orientierte. Die Fragen gliederten sich in die Themenbereiche OOP-Lehre, typische Lernschwierigkeiten von Studierenden sowie Anforderungen an ein Serious Game zur Vermittlung objektorientierter Programmierung. Die Interviews dauerten jeweils etwa 20 bis 30 Minuten.

Das Interview mit Experte A wurde digital aufgezeichnet, automatisiert transkribiert und anschließend hinsichtlich offensichtlicher Transkriptionsfehler sprachlich bereinigt. Für das Interview mit Experte B liegt keine vollständige Transkription vor. Die Auswertung dieses Interviews basiert daher auf den während des Gesprächs angefertigten Notizen. Zur Herstellung der Nachvollziehbarkeit befinden sich das bereinigte Transkript von Experte A sowie die strukturierten Gesprächsnotizen von Experte B im Anhang. Der verwendete Interviewleitfaden ist in Anhang A dokumentiert.

Die Auswertung erfolgte thematisch entlang der zentralen Themenbereiche des Interviewleitfadens. Dabei wurden sowohl wiederkehrende Schwerpunkte als auch Unterschiede zwischen den Aussagen der beiden Experten herausgearbeitet.

3.2.2 Ergebnisse

Im Folgenden werden die Ergebnisse der thematischen Analyse der Experteninterviews dargelegt. Zunächst werden relevante OOP-Konzepte betrachtet, anschließend typische Lernschwierigkeiten und schließlich die Anforderungen an ein Serious Game zur Vermittlung objektorientierter Programmierung.

Wichtige OOP-Konzepte

Beide Experten messen den grundlegenden Konzepten der Programmierung und insbesondere der Unterscheidung zwischen Klassen und Objekten eine hohe Bedeutung bei (vgl. Anhang B.1.3 und B.2.3). Experte A hebt darüber hinaus die Objekterzeugung, die Kommunikation zwischen Objekten, Konstruktoren sowie die Unterscheidung zwischen Compile-Zeit und Laufzeit hervor (vgl. Anhang B.1.3).

Experte B ergänzt zusätzlich weiterführende Konzepte wie Datenkapselung, Vererbung, Polymorphie, Interfaces, Komposition, Typsicherheit und Generics (vgl. Anhang B.2.10). Für eine spielerische Vermittlung sieht Experte B insbesondere in visuellen Lernformen Potenzial (vgl. Anhang B.2.8), während Experte A unter anderem die Visualisierung zeitlicher Abläufe wie Compile-Zeit und Laufzeit als einen möglichen Ansatz nennt (vgl. Anhang B.1.8).

Lernschwierigkeiten

Beide Experten berichten von wiederkehrenden Verständnisproblemen bei grundlegenden Programmier- und OOP-Konzepten. Insbesondere die Unterscheidung zwischen Klassen und Objekten wird von beiden als wiederkehrende Schwierigkeit beschrieben (vgl. Anhang B.1.6 und B.2.7).

Experte A weist darüber hinaus auf Defizite bei Konstruktoren, Interfaces und abstrakten Klassen sowie auf fehlende Grundlagen zur Unterscheidung von Compile-Zeit und Laufzeit hin (vgl. Anhang B.1.2 und B.1.6). Laut seiner Einschätzung erschweren Probleme in grundlegenden Programmierkenntnissen zunehmend die Vermittlung darauf aufbauender objektorientierter Konzepte.

Experte B beschreibt zusätzlich Schwierigkeiten bei Datenkapselung, Polymorphie und Vererbung sowie bei der sinnvollen Verteilung von Programmlogik auf unterschiedliche Klassen (vgl. Anhang B.2.3). Als grundlegende Ursache werden unter anderem heterogene Vorkenntnisse, begrenzte Übungszeit und teilweise zu geringe eigenständige Beschäftigung mit Programmierung genannt (vgl. Anhang B.2.2 und B.2.7).

Beide Interviews unterstreichen damit die Bedeutung praktischer Auseinandersetzung mit Programmierkonzepten. Experte A bezeichnet das eigenständige Ausprobieren ausdrücklich als wesentlichen Bestandteil des Programmierlernens (vgl. Anhang B.1.4), während Experte B praktischen Übungen ebenfalls einen besonders hohen Stellenwert beimisst (vgl. Anhang B.2.5).

Anforderungen an ein Serious Game

Beide Experten sehen grundsätzlich Potenzial für den ergänzenden Einsatz von Serious Games zur Vermittlung objektorientierter Programmierung (vgl. Anhang B.1.7 und B.2.8). Zudem wurde übereinstimmend die Bedeutung von praktischen Übungen beziehungsweise die aktive Auseinandersetzung mit den Lerninhalten betont (vgl. Anhang B.1.4 und B.2.5). Für die Gestaltung des Spiels ergeben sich aus den Interviews mehrere konkrete Anforderungen. Experte A hebt unmittelbares Feedback beispielsweise in Form einer Bewertung oder ergänzender Hinweise hervor (vgl. Anhang B.1.11).

Beide Experten weisen auf die Bedeutung einer fachlich korrekten Verwendung der OOP-Begrifflichkeiten hin (vgl. Anhang B.1.8 und B.2.12). Auch ein möglichst einfacher Zugang über beispielsweise den Webbrowser wird von beiden als geeignete Option betrachtet (vgl. Anhang B.1.12 und B.2.12). Hinsichtlich der inhaltlichen Ausgestaltung setzt Experte A den Schwerpunkt vor allem auf grundlegende Inhalte wie unterschiedliche Objekte, Objekterzeugung und Attributzuweisung (vgl. Anhang B.1.9). Komplexere Inhalte wie die Factory Method und weitere Entwurfsmuster werden von ihm für den vorgesehenen Umfang als weniger geeignet eingeschätzt (vgl. Anhang B.1.10).

Experte B nennt zusätzlich Polymorphie, Vererbung, Interfaces, Komposition, Objektkommunikation und Datenkapselung als potenziell relevante Inhalte (vgl. Anhang B.2.10). Als schwieriger spielerisch vermittelbar werden unter anderem objektorientierte Analyse und Design, UML-Klassendiagramme sowie Relationen zwischen Klassen eingeschätzt. Funktionale Programmierung sollte bei einer Fokussierung auf OOP nicht zusätzlich behandelt werden (vgl. Anhang B.2.11).

3.3 Zusammenfassung der Anforderungsanalyse

Die Literaturrecherche und die Experteninterviews zeigen sowohl übereinstimmende als auch ergänzende Anforderungen an die Gestaltung von *DSD*. Während die Literatur allgemeine Gestaltungsprinzipien für Serious Games liefert, konkretisieren die Experten diese im Hinblick auf die Vermittlung objektorientierter Programmierung. Tabelle 3.1 führt die daraus abgeleiteten Anforderungen zusammen und kennzeichnet deren jeweilige Grundlage. Der Anforderungskatalog bildet anschließend die Grundlage für die Konzeption von *DSD* in Kapitel 4 und konkretisiert zugleich die Rahmenbedingungen, unter denen die Forschungsfrage im weiteren Verlauf der Arbeit untersucht wird.

Abgeleitete Anforderungen	Grundlage
Schrittweise Vermittlung der Lerninhalte	Literatur, Experteninterviews
Verknüpfung von Lern- und Spielzielen	Literatur
Unmittelbares Feedback	Literatur, Experteninterviews
Aktive Anwendung der Lerninhalte	Literatur, Experteninterviews
Verwendung fachlich korrekter Begriffe	Experteninterviews
Fokus auf grundlegende OOP-Konzepte	Experteninterviews
Motivation durch Handlung und Progression	Literatur
Exploration und experimentelles Handeln	Literatur
Entscheidungen ohne reale Konsequenzen	Literatur
Intuitive und einfache Bedienung	Literatur
Zielgruppenorientierte Gestaltung	Literatur
Überprüfung der pädagogischen Wirksamkeit	Literatur
Einfacher Zugang zum Spiel	Experteninterviews

TABELLE 3.1: Synthese der generellen Anforderungen an Serious Games aus der Literatur und der spezifischen Anforderungen an *DSD* aus den Experteninterviews

4 Konzeption von Deep Space Debugger

Die Konzeption von *Deep Space Debugger* basiert auf den in Kapitel 3 zusammengeführten Anforderungen sowie den in Kapitel 2 dargestellten fachlichen Grundlagen der objektorientierten Programmierung. Im Hinblick auf die Forschungsfrage wird ein Konzept entwickelt, das grundlegende OOP-Konzepte in spielerische Interaktionen und Aufgaben eines 2D Serious Games überträgt. Das folgende Kapitel beschreibt die daraus abgeleiteten Lernziele, das Spielkonzept, die didaktische Gestaltung sowie die technische Konzeption.

4.1 Lernziele

Die Lernziele von *DSD* orientieren sich an den Erkenntnissen der fachlichen Grundlagen (siehe Kapitel 2) sowie der Anforderungsanalyse (siehe Kapitel 3). Im Mittelpunkt steht die Vermittlung grundlegender Konzepte der objektorientierten Programmierung, die sowohl aufgrund ihrer fachlichen Bedeutung als auch aufgrund identifizierter Lernschwierigkeiten ausgewählt wurden. Die Auswahl der konkreten Lernziele orientiert sich dabei an den in Abschnitt 2.1.4 dargestellten Grundprinzipien der objektorientierten Programmierung und den in der Anforderungsanalyse identifizierten Lernschwierigkeiten. Die daraus abgeleiteten Lernziele sowie deren Zuordnung zu den jeweiligen Spielbereichen sind in Tabelle 4.1 dargestellt. Die Spielenden sollen die Konzepte nicht ausschließlich kennenlernen, sondern deren grundlegende Zusammenhänge durch die Interaktion mit der Spielwelt nachvollziehen können.

Das Serious Game verfolgt das Ziel, den Spielenden ein grundlegendes Verständnis zentraler Konzepte des objektorientierten Paradigmas zu vermitteln. Die Konzepte werden innerhalb des Spiels schrittweise und aufeinander aufbauend eingeführt und in konkrete Spielhandlungen integriert. Die Reihenfolge der Konzeptvermittlung orientiert sich an einem steigenden Komplexitätsgrad. Zunächst werden grundlegende Konzepte wie Klassen und Objekte eingeführt, bevor darauf aufbauende Prinzipien wie Kapselung, Vererbung,

Objektkommunikation und Polymorphie vermittelt werden. Dadurch soll eine Überforderung der Spielenden vermieden und ein kontinuierlicher Wissensaufbau ermöglicht werden.

Fachliches Lernziel	Umsetzung im Spiel
Klassen und Objekte unterscheiden und deren Zusammenhang nachvollziehen	Energieversorgung
Prinzip der Kapselung und den Zugriff über definierte Schnittstellen nachvollziehen	Sicherheitssystem
Gemeinsame und spezialisierte Eigenschaften durch Vererbung erkennen	Robotik
Kommunikation zwischen Objekten durch Methodenaufrufe nachvollziehen	Stationsserver
Unterschiedliches Verhalten von Objekten bei einer gemeinsamen Schnittstelle erkennen	KI-Kern

TABELLE 4.1: Zuordnung der fachlichen Lernziele zu den Spielbereichen

Neben den fachlichen Lernzielen verfolgt *DSD* das übergeordnete didaktische Ziel, die ausgewählten OOP-Konzepte durch aktive Interaktion erfahrbar zu machen. Die Spielenden sollen die vermittelten Zusammenhänge nicht ausschließlich über textbasierte Erklärungen kennenlernen, sondern diese durch Exploration, Problemlösung und die Interaktion mit den Systemen der Spielwelt nachvollziehen und anwenden. Dadurch soll ein aktiver und experimenteller Lernprozess gefördert werden. Gleichzeitig soll die Vermittlung ohne umfangreiche Programmierkenntnisse oder klassische Codeeingabe erfolgen, um einen möglichst zugänglichen Einstieg in die grundlegenden Konzepte der objektorientierten Programmierung zu ermöglichen. Die konkrete Umsetzung dieser didaktischen Zielsetzung wird in Abschnitt 4.3 erläutert.

4.2 Spielkonzept

Das Spielkonzept beschreibt den grundlegenden Aufbau und Ablauf von *Deep Space Debugger*. Im Mittelpunkt steht die Verbindung einer narrativen Spielwelt mit spielerischen Herausforderungen und der Vermittlung objektorientierter Programmierung. Die Lerninhalte werden unmittelbar in die Spielmechaniken eingebettet, sodass die Auseinandersetzung mit den OOP-Konzepten als Bestandteil des Gameplays erfolgt. Damit wird insbesondere die in der Anforderungsanalyse identifizierte Verknüpfung von Lern- und Spielzielen aufgegriffen (siehe Tabelle 3.1).

4.2.1 Spielprinzip

DSD ist ein prototypisches zweidimensionales Serious Game im Science-Fiction-Setting einer beschädigten Raumstation. Die Spielenden übernehmen die Rolle eines Systemtechnikers, der nach einem schwerwiegenden Systemausfall verschiedene Bereiche der Station wiederherstellen muss. Dabei werden technische Probleme analysiert und mithilfe objektorientierter Denkweisen gelöst.

Das Spiel verbindet Gameplay, Rätseldesign und didaktische Lernmechaniken. Die Vermittlung der Inhalte erfolgt nicht über klassische Quellcodeeingabe oder isolierte Quizfragen, sondern durch interaktive Aufgaben und Spielmechaniken, die zentrale Konzepte der objektorientierten Programmierung repräsentieren. Damit werden insbesondere die Anforderungen einer aktiven Anwendung der Lerninhalte sowie der Verbindung von Lern- und Spielzielen aufgegriffen (siehe Tabelle 3.1). Die visuelle Gestaltung basiert auf einer stilisierten zweidimensionalen Pixelgrafik. Atmosphärische Audioelemente sowie eine reduzierte Benutzeroberfläche ergänzen die Spielwelt und unterstützen die Orientierung der Spielenden.

4.2.2 Spielwelt und Handlung

DSD spielt auf einer beschädigten Raumstation im tiefen Weltraum. Nach einem schwerwiegenden Systemausfall sind zentrale Bereiche der Station außer Betrieb. Die Spielenden übernehmen die Rolle eines Systemtechnikers, der als Notfallmaßnahme vom System aus dem Kryoschlaf geweckt wird, um die Ursache der Störungen zu untersuchen und die verschiedenen Stationssysteme schrittweise wiederherzustellen.

Das Science-Fiction-Setting wurde gewählt, da die technischen Systeme der Raumstation eine direkte Einbettung der vermittelnden OOP-Konzepte in die Spielwelt ermöglichen. Roboter, Terminals, Sicherheitssysteme und Energieanlagen dienen dabei als konkrete

Repräsentationen abstrakter objektorientierter Strukturen und Interaktionen. Die Wiederherstellung der einzelnen Stationsbereiche verbindet dadurch den narrativen Spielfortschritt mit der Vermittlung der jeweiligen Lerninhalte.

4.2.3 Spielmechaniken

Die Spielmechaniken bilden die Grundlage der Interaktion zwischen den Spielenden und der Spielwelt. Ihre Gestaltung berücksichtigt insbesondere die in der Anforderungsanalyse identifizierten Anforderungen der aktiven Anwendung von Lerninhalten, der Exploration sowie des experimentellen Handelns (siehe Tabelle 3.1). Durch die Exploration erschließen die Spielenden schrittweise die verschiedenen Bereiche der Raumstation. Neue Bereiche werden durch das erfolgreiche Abschließen vorheriger Aufgaben freigeschaltet und erweitern damit kontinuierlich den zugänglichen Spielraum.

Eine weitere zentrale Mechanik bildet die Interaktion mit den Objekten und technischen Systemen. Gegenstände können aufgenommen, transportiert und an vorgesehenen Stellen eingesetzt werden. Dazu gehören beispielsweise Energiekern- oder Sicherheitsmodule. Schalter, Terminals und weitere Bedienelemente ermöglichen die Aktivierung und Steuerung verschiedener Systeme innerhalb der Raumstation.

Die technischen Störungen der Raumstation bilden den Ausgangspunkt der spielerischen Problemstellungen. Zur Lösung interagieren die Spielenden mit den jeweiligen Systemen und wenden die im Spiel repräsentierten objektorientierten Konzepte an. Dazu gehören unter anderem die Wiederherstellung der Energieversorgung, die Interaktion mit Sicherheitssystemen und der Einsatz verschiedener Robotertypen.

Das Missionssystem strukturiert den Lern- und Spielverlauf durch aufeinanderfolgende Haupt- und Teilaufgaben. Es zeigt den aktuellen Fortschritt an und führt die Spielenden durch die einzelnen Bereiche der Raumstation. Damit unterstützt es zugleich die Anforderungen einer schrittweisen Vermittlung und eines unmittelbaren Feedbacks.

4.2.4 Core Gameplay Loop

Der Core Gameplay Loop (siehe Abbildung 4.1) beschreibt den wiederkehrenden Ablauf der zentralen Spielaktivitäten in *DSD* und strukturiert den kontinuierlichen Spielfortschritt. Er verbindet Exploration, Problemanalyse, die Anwendung objektorientierter Konzepte und die daraus resultierende Progression zu einem wiederkehrenden Spielzyklus.

Jede abgeschlossene Spielphase führt dabei zur nächsten Herausforderung und ermöglicht den schrittweisen Fortschritt innerhalb der Raumstation. Der Spielzyklus beginnt mit der Erkundung eines Stationsbereichs, in dem die Spielenden auf ein technisches Problem oder eine Fehlfunktion treffen.

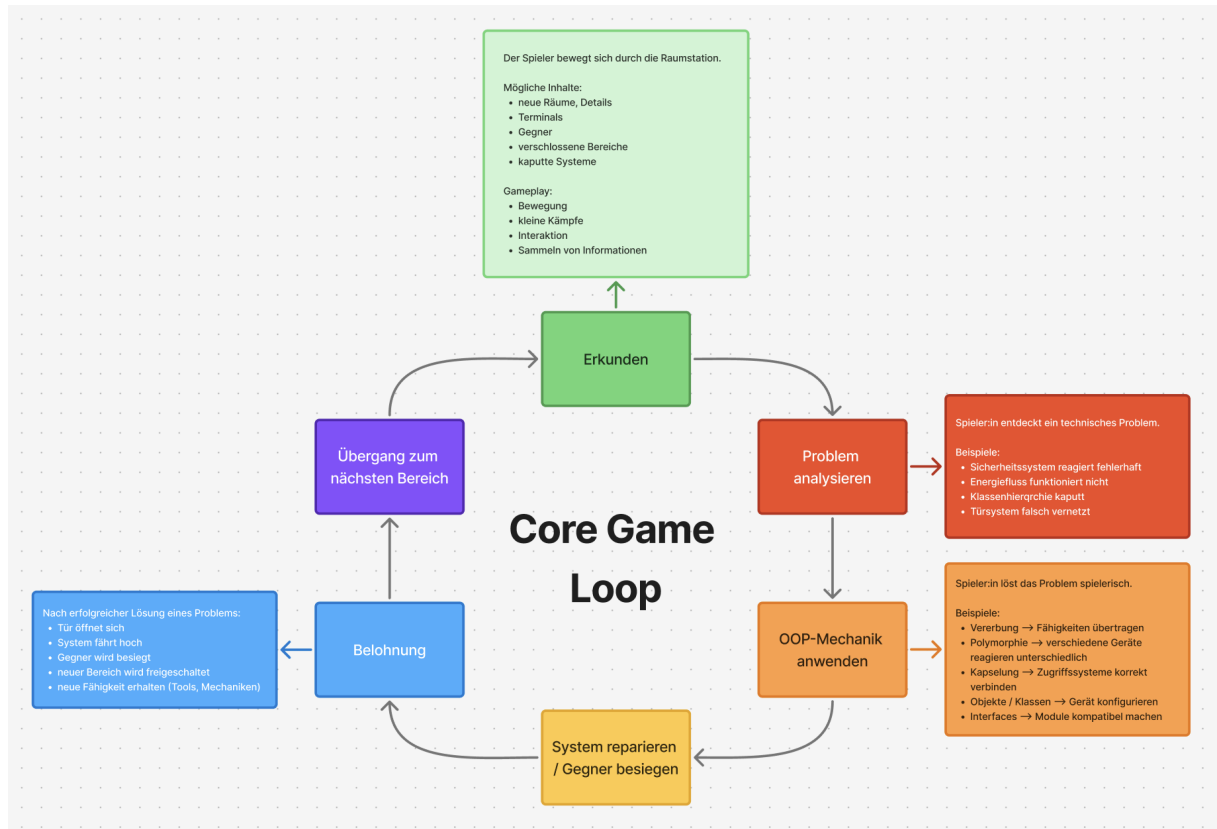


ABBILDUNG 4.1: Core Gameplay Loop von *Deep Space Debugger*

Anschließend wird das Problem analysiert, und es werden die für dessen Lösung erforderlichen Informationen gesammelt. Durch die Interaktion mit Objekten, Systemen oder Rätseln wird das jeweilige objektorientierte Konzept angewendet und das betroffene System repariert beziehungsweise aktiviert. Nach erfolgreicher Problemlösung erhalten die Spielenden unmittelbares visuelles und akustisches Feedback sowie Zugang zu weiteren Bereichen der Raumstation. Mit einer neuen Problemstellung beginnt der Spielzyklus erneut. Der Core Gameplay Loop verbindet damit die in der Anforderungsanalyse identifizierten Elemente der Exploration, aktiven Anwendung, unmittelbaren Rückmeldung und Progression innerhalb eines wiederkehrenden Spielablaufs.

4.2.5 Spielablauf

Der Spielablauf von *DSD* folgt einer linearen Progression. Die einzelnen Spielabschnitte greifen die in Abschnitt 2.1.4 dargestellten Konzepte der objektorientierten Programmierung sowie die daraus abgeleiteten Lernziele aus Abschnitt 4.1 auf und übertragen diese in konkrete Spielhandlungen. Den Einstieg bildet ein Tutorial im Aufwachraum der Raumstation. Dort werden die Ausgangssituation der Handlung, die grundlegende Steuerung sowie erste Interaktionsmöglichkeiten vermittelt. Gleichzeitig erhalten die Spielenden einen Überblick über die Benutzeroberfläche und die zentralen Spielmechaniken.

Im anschließenden Bereich der Energieversorgung werden Klassen und Objekte thematisiert. Die Spielenden analysieren verschiedene Energiekerne und deren Eigenschaften und setzen einen geeigneten Energiekern zur Wiederherstellung der Energieversorgung ein. Nach erfolgreicher Reaktivierung wird der Zugang zum nächsten Stationsbereich freigeschaltet.

Im Bereich der Sicherheitssysteme steht die Kapselung im Mittelpunkt. Verschlussene Türen und geschützte Systeme können nicht unmittelbar manipuliert werden, sondern müssen über dafür vorgesehene Schnittstellen beziehungsweise Kontrollsysteme angesprochen werden.

Der Robotikbereich vermittelt anschließend das Prinzip der Vererbung. Verschiedene Robotertypen besitzen gemeinsame Eigenschaften und Verhaltensweisen, unterscheiden sich jedoch hinsichtlich ihrer jeweiligen Spezialisierungen. Die Spielenden müssen diese Unterschiede erkennen und die jeweiligen Fähigkeiten zur Lösung der Aufgaben einsetzen.

Im Kommunikationszentrum wird die Kommunikation zwischen Objekten thematisiert. Durch die Interaktion mit verschiedenen Systemen werden Nachrichten beziehungsweise Methodenaufrufe zwischen den Objekten und dem Stationsserver ausgelöst.

Den Abschluss bildet der KI-Kern, in dem das Konzept der Polymorphie aufgegriffen wird. Derselbe Methodenaufwurf führt bei unterschiedlichen Objekten zu jeweils spezifischen Verhaltensweisen. Die erfolgreiche Reaktivierung des KI-Kerns stellt das finale Ereignis des Prototyps dar und schließt die Wiederherstellung der Raumstation ab.

4.3 Didaktische Gestaltung

Die didaktische Gestaltung von *DSD* orientiert sich an den in Kapitel 3 zusammengeführten Anforderungen. Im Mittelpunkt stehen dabei insbesondere die schrittweise Vermittlung, die aktive Anwendung der Lerninhalte, die Verknüpfung von Lern- und Spielzielen sowie unmittelbares Feedback (siehe Tabelle 3.1). Ziel ist es, die zentralen Konzepte der objektorientierten Programmierung nicht isoliert zu erläutern, sondern durch spielerische Interaktion, schrittweise Progression und unmittelbares Feedback erfahrbar zu machen. Die Vermittlung orientiert sich dabei an den in Abschnitt 2.2 dargestellten Prinzipien des Game-Based Learning, nach denen der Wissenserwerb eng mit dem Spielgeschehen verknüpft ist.

4.3.1 Vermittlung der OOP-Konzepte

Die Vermittlung der objektorientierten Programmierung erfolgt in *DSD* nicht anhand einer konkreten Programmiersprache, sondern auf konzeptioneller Ebene. Die in Abschnitt 2.1.4 beschriebenen Konzepte werden hierzu auf konkrete Elemente und Handlungen innerhalb der Spielwelt übertragen. Im Hinblick auf die Forschungsfrage bildet diese Übertragung den zentralen Ansatz, um abstrakte OOP-Zusammenhänge innerhalb eines 2D Serious Games verständlich und interaktiv erfahrbar zu machen.

Die didaktische Gestaltung basiert auf der Übertragung abstrakter OOP-Konzepte auf visuell und interaktiv erfahrbare Zusammenhänge. Klassen, Objekte, Zustände, Schnittstellen und Methodenaufrufe werden nicht ausschließlich textuell beschrieben, sondern durch Objekte und Systeme innerhalb der Raumstation repräsentiert. Die Spielenden erschließen die zugrunde liegenden Zusammenhänge und Konzepte durch die Interaktion mit diesen Elementen und wenden sie zur Lösung konkreter Aufgaben an.

Die Lerninhalte sind dabei unmittelbar mit den Spielhandlungen verknüpft. Das erfolgreiche Anwenden eines OOP-Konzepts führt gleichzeitig zum Fortschritt innerhalb der Spielwelt. Dadurch wird die in der Anforderungsanalyse formulierte Verknüpfung von Lern- und Spielzielen konzeptionell umgesetzt, indem der fachliche Lernprozess unmittelbar mit dem Spielfortschritt verbunden wird. Die konkrete Zuordnung der einzelnen OOP-Konzepte zu den jeweiligen Spielbereichen und Aufgaben wurde bereits im Spielablauf (siehe Abschnitt 4.2.5) dargestellt.

4.3.2 Progression des Lernprozesses

Der Lernprozess in *DSD* greift die in der Anforderungsanalyse abgeleitete schrittweise Vermittlung der Lerninhalte auf und orientiert sich zugleich an den in Abschnitt 2.2 dargestellten Prinzipien des GBL. Neue Lerninhalte werden schrittweise eingeführt und bauen auf bereits erworbenem Wissen auf. Das Tutorial dient zunächst dazu, die Spielenden mit der Steuerung, den grundlegenden Spielmechaniken sowie den Interaktionsmöglichkeiten vertraut zu machen. Erst anschließend beginnt die Vermittlung der objektorientierten Konzepte, sodass sich die Spielenden auf die fachlichen Lerninhalte konzentrieren können. Die Vermittlung der OOP-Konzepte erfolgt progressiv. Jeder Spielbereich führt ein neues Konzept ein, während bereits behandelte Inhalte in späteren Abschnitten erneut aufgegriffen und mit neuen Zusammenhängen verknüpft werden. Das Missionssystem unterstützt diese Progression, indem Haupt- und Untermissionen in einer festgelegten Reihenfolge freigeschaltet werden. Einzelne Lernabschnitte können dadurch nicht übersprungen werden, wodurch ein strukturierter und nachvollziehbarer Lernprozess entsteht.

Entsprechend den Prinzipien des GBL erfolgt der Wissenserwerb überwiegend durch aktives Handeln innerhalb der Spielwelt. Die Spielenden lösen eigenständig Aufgaben, erkunden die Raumstation und interagieren mit verschiedenen Objekten und Systemen. Die fachlichen Inhalte werden dadurch nicht ausschließlich theoretisch vermittelt, sondern unmittelbar angewendet und erfahrbar gemacht. Im Mittelpunkt steht das eigenständige Problemlösen anstelle des reinen Auswendiglernens von Begrifflichkeiten.

Die Gestaltung der Lernprogression lässt sich zudem in das in Abschnitt 2.4 dargestellte MDA-Framework einordnen. Das Missionssystem und die schrittweise Freischaltung neuer Inhalte stellen zentrale *Mechanics* dar. Durch deren Zusammenspiel mit den Handlungen der Spielenden entstehen *Dynamics* wie Exploration und Problemlösung. Die daraus resultierenden Erfolgserlebnisse und das Wahrnehmen des eigenen Fortschritts tragen zur angestrebten Spielerfahrung (*Aesthetics*) bei.

4.3.3 Feedback und Motivation

Feedback stellt entsprechend der in der Anforderungsanalyse identifizierten Anforderung einer unmittelbaren Rückmeldung einen zentralen Bestandteil des Lernprozesses in *DSD* dar. Die Spielenden erhalten während des gesamten Spielverlaufs Rückmeldungen über ihren Fortschritt und ihre Interaktionen. Dadurch sollen erfolgreiche Aktionen bestätigt, Fehlhandlungen erkennbar gemacht und die Bearbeitung der Aufgaben unterstützt werden. Die Rückmeldungen erfolgen über verschiedene Elemente des Spiels. Das Missionssystem visualisiert den aktuellen Fortschritt und zeigt den erfolgreichen Abschluss einzelner

Aufgaben an. Dialoge vermitteln zusätzliche Hinweise und unterstützen die Orientierung innerhalb der Spielwelt. Ergänzend werden visuelle Hervorhebungen interaktiver Objekte sowie akustische Signale eingesetzt, um relevante Aktionen und Ereignisse unmittelbar kenntlich zu machen.

Neben dem Feedback stellt die Motivation einen weiteren Bestandteil der didaktischen Gestaltung dar. Die fortlaufende Handlung, die schrittweise Freischaltung neuer Spielbereiche sowie der sichtbare Missionsfortschritt greifen dabei die in der Anforderungsanalyse identifizierte Motivation durch Handlung und Progression auf. Entsprechend den in Abschnitt 2.2 dargestellten Prinzipien des GBL sollen die Spielenden zur aktiven Auseinandersetzung mit den Aufgaben und zur eigenständigen Exploration der Spielwelt angeregt werden. Die fortlaufende Handlung, die schrittweise Freischaltung neuer Spielbereiche sowie der sichtbare Missionsfortschritt schaffen hierfür kontinuierliche Zwischenziele und machen den eigenen Fortschritt nachvollziehbar.

Die Gestaltung lässt sich zudem in das in Abschnitt 2.4 dargestellte MDA-Framework einordnen. Das Missionssystem und die verschiedenen Feedbackmechanismen stellen *Mechanics* dar, aus deren Zusammenspiel mit den Handlungen der Spielenden *Dynamics* wie Exploration und Problemlösung entstehen. Die daraus resultierenden Erfolgserlebnisse und das Wahrnehmen des eigenen Fortschritts sollen zu einer motivierenden Spielerfahrung (*Aesthetics*) beitragen.

4.4 Technische Konzeption

Die technische Konzeption von *DSD* umfasst die Auswahl geeigneter Entwicklungswerkzeuge, grundlegende Entscheidungen hinsichtlich der Softwarearchitektur und der Gestaltung der Benutzeroberfläche. Ziel ist eine technische Struktur, die eine effiziente Umsetzung des Prototyps ermöglicht und zugleich dessen Erweiterbarkeit und Bedienbarkeit unterstützt.

4.4.1 Wahl der Entwicklungsumgebung

Für die Entwicklung des Prototyps wurde eine Game Engine benötigt, die insbesondere eine umfangreiche 2D-Unterstützung, eine objektorientierte Skriptsprache sowie eine modulare und erweiterbare Entwicklung ermöglicht.

Die Wahl fiel auf die Open-Source-Game-Engine *Godot*. Ausschlaggebend waren insbesondere die integrierte Unterstützung für 2D-Spiele sowie die szenen- und nodebasierte Struktur der Engine. Spielobjekte und Spielsysteme können dadurch modular aufgebaut und miteinander verknüpft werden. Dies unterstützt die für den Prototyp angestrebte Trennung der verschiedenen Spielsysteme.[30]

Als Skriptsprache wird *GDScript* eingesetzt. Diese ist unmittelbar in Godot integriert und unterstützt objektorientierte Konzepte wie Klassen und Vererbung.[30] Die Verwendung einer objektorientierten Skriptsprache bietet sich insbesondere vor dem Hintergrund der inhaltlichen Ausrichtung von *DSD* auf das objektorientierte Paradigma an.

Neben der Entwicklungsumgebung wurde auch die Zielplattform des Prototyps berücksichtigt. Um einen möglichst einfachen Zugang zum Serious Game zu ermöglichen, ist *DSD* als browserbasierte Anwendung vorgesehen. Diese Entscheidung greift die in der Anforderungsanalyse zusammengeführte Anforderung eines Zugangs zum Spiel auf (siehe Tabelle 3.1). Die Bereitstellung als Webanwendung ermöglicht die Nutzung des Prototyps ohne vorherige Installation und reduziert die technischen Einstiegshürden für die Spielenden. Die Godot Engine unterstützt hierfür den Export als Webanwendung.[30]

4.4.2 Softwarearchitektur

Für *DSD* wurde eine modulare Softwarearchitektur vorgesehen, bei der die verschiedenen Funktionen des Spiels in weitgehend eigenständige Systeme aufgeteilt werden. Dadurch sollen einzelne Komponenten unabhängig entwickelt, angepasst und erweitert werden können.

Zu den zentral vorgesehenen Komponenten gehören unter anderem das Missions- und Dialogsystem, das Inventar, die Minimap sowie das Terminalsystem. Die einzelnen Systeme übernehmen jeweils klar abgegrenzte Verantwortlichkeiten und sollen nur über definierte Schnittstellen miteinander kommunizieren. Dadurch wird eine möglichst geringe Abhängigkeit zwischen den Komponenten angestrebt.

Der modulare Aufbau soll insbesondere die Erweiterung des Prototyps unterstützen. Zusätzliche Spielbereiche, Interaktionsobjekte und Mechaniken können dadurch ergänzt werden, ohne grundlegende Änderungen an der gesamten Spielstruktur vornehmen zu müssen. Die konkrete technische Umsetzung der einzelnen Systeme wird in Kapitel 5 erläutert.

4.4.3 Gestaltung der Benutzeroberfläche

Die Benutzeroberfläche von *DSD* wurde mit dem Ziel gestaltet, die Spielenden bei der Orientierung und Bearbeitung der Aufgaben zu unterstützen, ohne das eigentliche Spielgeschehen unnötig zu überlagern. Entsprechend der in Abschnitt 3.1 abgeleiteten Anforderung einer intuitiven und einfachen Bedienung folgt das Interface einem reduzierten und übersichtlichen Gestaltungskonzept.

Zentrale Informationen wie aktuelle Missionsziele oder Interaktionshinweise sollen unmittelbar in der Spielansicht verfügbar sein. Ergänzend sind ein Inventar, eine Minimap sowie verschiedene Terminaloberflächen vorgesehen, über die zusätzliche Informationen und Interaktionsmöglichkeiten bereitgestellt werden. Die Missionsanzeige dient insbesondere dazu, den aktuellen Fortschritt sichtbar und die jeweils nächste Aufgabe nachvollziehbar zu machen. Die grafische Gestaltung der Benutzeroberfläche orientiert sich am Pixelart-Stil der Spielwelt, um ein einheitliches Erscheinungsbild zu gewährleisten. Gleichzeitig sollen eine klare Anordnung und die Beschränkung auf relevante Informationen verhindern, dass die Benutzeroberfläche selbst zu einer zusätzlichen Hürde für die Spielenden wird.

5 Implementierung

In diesem Kapitel wird die technische Umsetzung des zuvor konzipierten Serious Games *DSD* beschrieben. Aufbauend auf den in Kapitel 4 dargestellten Lernzielen, Spielmechaniken sowie didaktischen und technischen Konzeptentscheidungen werden zunächst die eingesetzten Entwicklungswerkzeuge vorgestellt. Anschließend wird die Implementierung der zentralen Spielsysteme erläutert, bevor die technische Umsetzung der Benutzeroberfläche betrachtet wird. Der Fokus liegt auf den wesentlichen Komponenten des Prototyps, die für den Spielablauf und die Vermittlung der Lerninhalte relevant sind.

5.1 Entwicklungsumgebung

Die Entwicklung des Prototyps erfolgte entsprechend der in Abschnitt 4.4.1 getroffenen Entscheidung mit der Open-Source-Game-Engine *Godot* in der Version 4.7. Als Programmiersprache wurde die in Godot integrierte Skriptsprache *GDScript* verwendet. Während der Entwicklung wurde der Prototyp primär als Desktop-Anwendung umgesetzt. Zusätzlich ermöglicht Godot den Export als Webanwendung, wodurch *DSD* ohne lokale Installation über einen Webbrowser ausgeführt werden kann. Ergänzend wurden grundlegende Touch-Bedienelemente für mobile Endgeräte integriert, darunter virtuelle Steuerelemente für die Bewegung und Interaktion.

Abbildung 5.1 zeigt exemplarisch die Entwicklungsumgebung während der Umsetzung von *DSD*. Neben dem zentralen Arbeitsbereich sind der Szenenbaum zur hierarchischen Organisation der Nodes sowie die im Projekt verwendeten Ordnerstrukturen dargestellt. Die einzelnen Spielbereiche und Spielsysteme werden innerhalb von Godot als Szenen und Nodes organisiert und über zugehörige GDScript-Dateien um ihre jeweilige Spiellogik ergänzt. Für die Versionsverwaltung und Projektorganisation wurde das Versionskontrollsystem *Git* in Verbindung mit *GitHub* eingesetzt. Dadurch konnten die Entwicklungsstände nachvollzogen, Änderungen versioniert und der Quellcode zentral verwaltet werden.

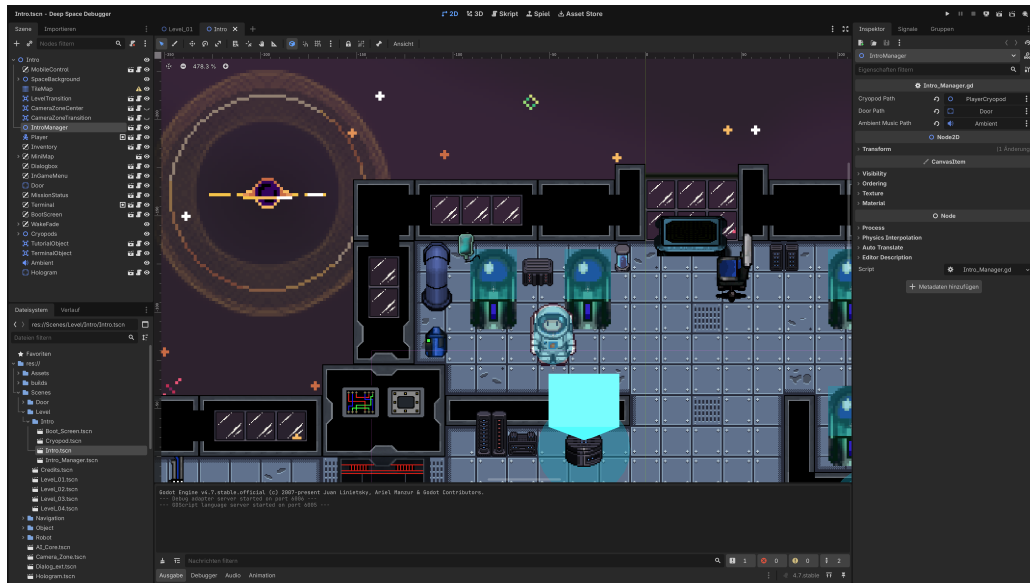


ABBILDUNG 5.1: Entwicklungsumgebung von *DSD* in der Godot Engine

Die grafische Gestaltung des Prototyps basiert auf einem Pixelart-Stil. Für den Aufbau der Spielwelt wurden überwiegend frei verfügbare Assets von Plattformen wie *itch.io* verwendet und bei Bedarf an das Gestaltungskonzept angepasst. Zur Bearbeitung einzelner Grafiken und Sprites kamen *Aseprite* und *LibreSprite* zum Einsatz. Durch die Nutzung bereits bestehender Assets konnte der Entwicklungsaufwand insbesondere bei der Gestaltung der Spielwelt reduziert werden.

Für die akustische Gestaltung wurden frei verfügbare Soundeffekte, Musik und Hintergrundgeräusche aus Soundbibliotheken wie *Freesound* und *Pixabay* verwendet. Diese unterstützen sowohl die akustische Rückmeldung auf Interaktionen als auch die atmosphärische Gestaltung der Spielwelt.

5.2 Umsetzung zentraler Spielsysteme

Die Umsetzung von *DSD* basiert auf mehreren eigenständigen Spielsystemen, die gemeinsam die in Kapitel 4 beschriebenen Spielmechaniken, den strukturierten Lernverlauf und die Vermittlung der OOP-Konzepte realisieren. Entsprechend der in Abschnitt 4.4 vorgesehenen modularen Softwarearchitektur wurden die Funktionen in weitgehend voneinander getrennten Komponenten umgesetzt. Im Folgenden werden die für den Prototyp zentralen Spielsysteme und deren technische Realisierung vorgestellt.

5.2.1 Spielverwaltung

Die Spielverwaltung übernimmt die Steuerung des übergeordneten Spielablaufs. Zu Beginn des Spiels wird zunächst eine Aufwachsequenz ausgeführt, welche die Spielenden in die Handlung einführt und in den Aufwachraum der Raumstation überleitet. Im Anschluss beginnt das Tutorial, in dem die grundlegenden Spielmechaniken, wie Bewegung, Interaktion und die Bedienung der Benutzeroberfläche schrittweise eingeführt werden. Die Steuerung des weiteren Spielfortschritts erfolgt über ein Missionssystem, dessen Zustände zentral durch einen Missionsmanager verwaltet werden. Sämtliche Missionen sind hierarchisch in Hauptmissionen und zugehörige Untermissionen gegliedert. Der Missionsmanager verwaltet deren jeweiligen Bearbeitungsstatus und schaltet bei erfolgreichem Abschluss einer Untermission die nachfolgende Aufgabe frei. Sind sämtliche Untermissionen abgeschlossen, wird die zugehörige Hauptmission als abgeschlossen markiert und die nächste Hauptmission aktiviert. Die Missionsdaten werden innerhalb des Missionsmanagers in einer strukturierten Datenstruktur gespeichert. Jede Hauptmission besitzt eine eindeutige Kennung, einen Bearbeitungsstatus sowie eine Liste der zugehörigen Untermissionen. Diese verfügen wiederum über eigene Statusinformationen und die für die Missionsanzeige benötigten Texte. Listing 5.1 zeigt exemplarisch den Aufbau einer Hauptmission mit den zugehörigen Untermissionen.

LISTING 5.1: Exemplarische Datenstruktur einer Hauptmission

```

" id": "tutorial",
" text": "Tutorial",
" done": false,
" subtasks": [
    {
        " text": "Press any movement key [WASD]",
        " done": false
    }
]

```


Die Aktualisierung des Missionsfortschritts erfolgt ereignisgesteuert. Wird eine Untermission erfolgreich abgeschlossen, sucht der Missionsmanager zunächst die zugehörige Hauptmission und Untermission, aktualisiert deren Bearbeitungsstatus und prüft anschließend, ob sämtliche Untermissionen abgeschlossen sind. In diesem Fall wird die Hauptmission über `complete_mission()` beendet. Listing 5.2 zeigt einen gekürzten Ausschnitt der hierfür implementierten Funktion.

LISTING 5.2: Gekürzte Aktualisierung des Missionsfortschritts

```
func complete_subtask(mission_id: String, subtask_id: String):
    for i in range(mission_steps.size()):
        var mission = mission_steps[i]
        if mission["id"] != mission_id:
            continue
        for subtask in mission["subtasks"]:
            if subtask["text"] == subtask_id:
                if subtask["done"]:
                    return
                subtask["done"] = true
                if i != current_mission_index:
                    return
                check_sound.play()
                await show_next_subtask_typed()
                update_minimap_target()
                if are_all_subtasks_done(mission):
                    await complete_mission(mission_id)
                return
```

Die Funktion `complete_subtask` wird von verschiedenen Spielsystemen aufgerufen, sobald eine missionsrelevante Aktion erfolgreich durchgeführt wurde. Dazu gehören beispielsweise Interaktionen mit Spielobjekten, die im nachfolgenden Abschnitt 5.2.2 näher erläutert werden. Durch die zentrale Verarbeitung im Missionsmanager bleibt die Verwaltung des Spielfortschritts von den jeweils auslösenden Spielobjekten getrennt.

5.2.2 Interaktionssystem

Das Interaktionssystem bildet die zentrale Schnittstelle zwischen den Spielenden und den interaktiven Objekten der Spielwelt. Hierfür wird ein einheitliches Interaktionskonzept

verwendet, bei dem Aktionen ausschließlich innerhalb eines definierten Interaktionsbereichs (**Area2D**) ausgeführt werden können. Sämtliche Interaktionen werden über die Taste **ENTER** ausgelöst, wodurch unabhängig vom jeweiligen Objekttyp ein konsistentes Bedienkonzept gewährleistet wird.

Damit wird die in Abschnitt 4.4.3 vorgesehene intuitive und einheitliche Bedienung technisch umgesetzt. Betreten die Spielenden den Interaktionsbereich eines Objekts, wird eine Referenz auf das entsprechende Objekt gespeichert und beim Verlassen des Bereichs wieder entfernt. Gleichzeitig wird das Objekt durch eine pulsierende Hervorhebung visuell gekennzeichnet und ein entsprechender Interaktionshinweis eingeblendet. Abbildung 5.2 zeigt diese Kennzeichnung exemplarisch anhand eines interaktiven Objekts.



ABBILDUNG 5.2: Visuelle Kennzeichnung eines interaktiven Objekts

Die technische Umsetzung der Reichweitenerkennung erfolgt über die Signale des jeweiligen **Area2D**-Bereichs. Wie Listing 5.3 exemplarisch zeigt, wird beim Betreten durch die Spielfigur der Interaktionsstatus gesetzt und eine Referenz auf diese gespeichert. Beim Verlassen des Bereichs werden beide Zustände wieder zurückgesetzt.

LISTING 5.3: Erkennung des Interaktionsbereichs eines Spielobjekts

```
func _on_body_entered(body):
    if body.name == "Player":
        player_nearby = true
        current_player = body
func _on_body_exited(body):
    if body.name == "Player":
        player_nearby = false
        current_player = null
```

Wird während einer gültigen Interaktionssituation die Interaktionstaste betätigt, wird die vorgesehene Funktion des aktuell referenzierten Objekts aufgerufen. Die konkrete Ausführung verbleibt dabei innerhalb des jeweiligen Spielobjekts. So kann beispielsweise eine Tür ihren Öffnungsvorgang ausführen, während ein aufnehmbares Objekt die Übergabe an das Inventarsystem auslöst. Das übergeordnete Interaktionssystem muss hierfür keine Kenntnis über die konkrete Funktion des jeweiligen Objekts besitzen.

Durch diese Trennung kann dasselbe Interaktionssystem für unterschiedliche Objekttypen verwendet werden. Neue interaktive Objekte können ergänzt werden, indem sie die vorgesehene Interaktionslogik bereitstellen, ohne dass die grundlegende Spielsteuerung angepasst werden muss. Dadurch wird die Erweiterbarkeit und Wiederverwendbarkeit des Systems unterstützt.

5.2.3 Inventar- und Objektsystem

Das Inventar- und Objektsystem ermöglicht die Aufnahme, Verwaltung und Verwendung transportierbarer Spielobjekte. Die Aufnahme erfolgt über das in Abschnitt 5.2.2 beschriebene Interaktionssystem. Um die Anzahl gleichzeitig transportierbarer Objekte zu begrenzen, besitzt das Inventar drei Inventarplätze. Die aktuelle Belegung wird den Spielenden dauerhaft über das HUD angezeigt (siehe Abbildung 5.3).



ABBILDUNG 5.3: Darstellung eines aufgenommen Objekts im Inventar

Jedes aufnehmbare Spielobjekt verfügt über eigene Objektdaten. Dazu gehören eine eindeutige Kennung, ein Name, die zugehörige Klasse sowie objektspezifische Attribute und die für die Darstellung verwendete Textur. Bei den im Bereich der Energieversorgung eingesetzten Energiekernen umfassen die Attribute beispielsweise Kapazität, Stabilität und Zustand. Die explizite Zuordnung einer Klasse sowie individueller Attributwerte bildet dabei die technische Grundlage für die in Abschnitt 4.2.5 beschriebene Vermittlung von Klassen und Objekten. Die Spielenden können verschiedene Objekte derselben Klasse anhand ihrer konkreten Attributwerte unterscheiden. Listing 5.4 zeigt exemplarisch die Definition dieser Daten innerhalb eines aufnehmbaren Objekts.

LISTING 5.4: Exemplarische Definition der Daten eines aufnehmbaren Objekts

```
@export var core_texture: AtlasTexture
@export var object_name := ""
@export var class_name_display := ""
@export var attributes := {
    "capacity": "0",
    "stability": "0",
    "state": "stable"
}
```

Bei erfolgreicher Aufnahme werden die relevanten Objektdaten an das Inventar übergeben, und das zugehörige Objekt in der Spielwelt ausgeblendet. Die dafür verwendete Funktion `get_inventory_data()` stellt die benötigten Informationen in einer gemeinsamen Datenstruktur bereit. Beim Ablegen wird das Objekt wieder in der Spielwelt positioniert und steht anschließend erneut für weitere Interaktionen zur Verfügung. Für missionsbezogene Aufgaben werden spezielle Ablagebereiche in Form von Dropzones realisiert. Wird ein Objekt in einer solchen Dropzone abgelegt, werden dessen Eigenschaften mit den für die Aufgabe definierten Anforderungen verglichen. Dadurch werden die dargestellten Objekteigenschaften unmittelbar in eine spielerische Aufgabe eingebunden: Die Spielenden müssen die Klasse und die konkreten Attributwerte der verfügbaren Objekte berücksichtigen, um das für die jeweilige Aufgabe geeignete Objekt auszuwählen. Damit wird die in Abschnitt ?? konzipierte aktive Anwendung der fachlichen Inhalte technisch unterstützt. Die Validierung erfolgt durch die Funktion `is_valid_item()`, deren zentrale Prüflogik in Listing 5.5 dargestellt ist.

LISTING 5.5: Validierung eines Objekts innerhalb einer Dropzone

```
func is_valid_item(item_class, item_attributes) -> bool:
    if item_class != required_class:
        return false
```

```

for key in required_attributes.keys():
    if !item_attributes.has(key):
        return false
    var required_value = required_attributes[key]
    var item_value = item_attributes[key]
    if str(item_value) != str(required_value):
        return false
return true

```

Nur bei einer gültigen Übereinstimmung wird das Objekt akzeptiert und der erfolgreiche Abschluss der entsprechenden Aufgabe an den in Abschnitt 5.2.1 beschriebenen Missionsmanager übermittelt. Dieser aktualisiert daraufhin den Missionsfortschritt. Abbildung 5.4 zeigt exemplarisch einen in einer Dropzone platzierten Energiekern.



ABBILDUNG 5.4: In einer Dropzone platzierter Energiekern

Durch die Trennung von Objektdaten, Inventarverwaltung, Dropzones und Missionsverwaltung bleiben die jeweiligen Verantwortlichkeiten weitgehend voneinander getrennt. Dadurch können weitere transportierbare Objekte und missionsspezifische Anforderungen ergänzt werden, ohne die grundlegenden Funktionen des Inventarsystems zu verändern.

5.2.4 Dialogsystem

Das Dialogsystem dient der Vermittlung von Handlungselementen, Hinweisen und aufgabenbezogenen Informationen während des Spielverlaufs. Es unterstützt die Orientierung

der Spielenden und stellt kontextbezogene Informationen unmittelbar innerhalb der Spielwelt bereit. Dialoge können durch unterschiedliche Ereignisse ausgelöst werden. Hierzu zählen insbesondere Interaktionen mit Spielobjekten, beispielsweise Hologrammen oder aufnehmbaren Objekten, sowie das Betreten definierter Bereiche (*Area2D*). Die jeweiligen Spielobjekte übergeben den anzuzeigenden Inhalt an das Dialogsystem, das im Anschluss die Darstellung innerhalb einer zentralen Dialogbox übernimmt. Dadurch kann dieselbe Benutzeroberfläche für unterschiedliche Dialogquellen und Spielsituationen wiederverwendet werden. Die Textausgabe erfolgt zeichenweise über einen Typewriter-Effekt und wird durch einen entsprechenden Soundeffekt akustisch begleitet. Neben allgemeinen Hinweisen können über die Dialogbox auch objektspezifische Informationen dargestellt werden. So werden beispielsweise bei der Untersuchung eines Energiekerns dessen Name, Klasse und Attribute aus den in Abschnitt 5.2.3 beschriebenen Objektdaten ausgelesen und innerhalb der Dialogbox angezeigt. Abbildung 5.5 zeigt die Darstellung von Informationen exemplarisch anhand eines Hologramms.



ABBILDUNG 5.5: Darstellung eines Dialogs über ein Hologramm

Durch die zentrale Dialogbox können unterschiedliche Spielobjekte Informationen bereitstellen, ohne jeweils eine eigene Benutzeroberfläche implementieren zu müssen. Gleichzeitig werden Hinweise und fachliche Informationen unmittelbar in der jeweiligen Spielsituation dargestellt und mit den entsprechenden Interaktionen innerhalb der Spielwelt verknüpft. Damit unterstützt das Dialogsystem die in Abschnitt 4.3 konzipierte Einbettung der Lerninhalte in das Spielgeschehen und setzt zugleich einen Teil der in Abschnitt 4.3.3 vorgesehenen Feedback- und Hinweismechanismen technisch um.

5.2.5 Tutorialsystem

Das Tutorialsystem wird im Einführungsbereich von *DSD* eingesetzt und dient der schrittweisen Einführung in die grundlegende Steuerung und die zentralen Spielmechaniken. Die einzelnen Tutorialaufgaben sind als Untermissionen in das im Abschnitt 5.2.1 beschriebene Missionssystem integriert und werden entsprechend der vorgesehenen Reihenfolge nacheinander freigeschaltet.

Die Tutorialaufgaben reagieren hierbei unmittelbar auf die Aktionen der Spielenden. Beispielsweise wird so die Verwendung der Bewegungssteuerung erkannt und die entsprechende Untermission nach erfolgreicher Ausführung als abgeschlossen markiert. Weitere Aufgaben führen schrittweise die Interaktion mit Spielobjekten, die Verwendung der Benutzeroberfläche sowie für den weiteren Spielverlauf relevante Funktionen ein. Der Abschluss einer Aufgabe wird an den Missionsmanager übermittelt, der den Missionsstatus aktualisiert und anschließend die nächste Tutorialaufgabe freigibt. Während des Tutorials erhalten die Spielenden kontinuierliches visuelles und akustisches Feedback. Der erfolgreiche Abschluss einer Tutorialaufgabe wird durch einen Bestätigungston sowie die Aktualisierung der Missionsanzeige signalisiert. Abbildung 5.6 zeigt exemplarisch die Darstellung der Tutorialaufgaben innerhalb der Benutzeroberfläche.



ABBILDUNG 5.6: Darstellung der Tutorialaufgaben innerhalb der Benutzeroberfläche

Erst nach Abschluss aller vorgesehenen Tutorialaufgaben wird der Übergang in den ersten regulären Spielbereich freigeschaltet. Dieses Vorgehen setzt die in Abschnitt 4.3.2

beschriebene Trennung zwischen der Einführung der Spielmechaniken und der anschließenden Vermittlung der fachlichen Lerninhalte technisch um.

5.2.6 Terminal- und Stationssystem

Das Terminalsystem bildet eine zentrale Schnittstelle zwischen den Spielenden und den verschiedenen technischen Systemen der Raumstation. Die Terminals werden über das in Abschnitt 5.2.2 beschriebene Interaktionssystem aktiviert und öffnen anschließend eine eigene Benutzeroberfläche. Die Navigation innerhalb des Terminalmenüs erfolgt über die Pfeiltasten, während die Auswahl eines Menüpunkts über die Interaktionstaste bestätigt wird. Die verfügbaren Funktionen unterscheiden sich abhängig vom jeweiligen Stationsbereich und dem aktuellen Spielfortschritt.

Die grundlegende Terminaloberfläche wird in mehreren Spielbereichen wiederverwendet und abhängig vom jeweiligen Level konfiguriert. Im Sicherheitsbereich werden beispielsweise Funktionen zur Erteilung einer Sicherheitsfreigabe bereitgestellt, während das Terminal im Robotikbereich die Auswahl und Ansteuerung verschiedener Robotertypen ermöglicht. Im Bereich des KI-Kerns werden wiederum Funktionen zur Aktivierung der dort vorgesehenen Methoden angeboten. Die levelabhängigen Terminalfunktionen greifen die in Abschnitt 4.2.5 beschriebenen OOP-spezifischen Spielhandlungen auf und übertragen die jeweiligen fachlichen Konzepte auf konkrete Interaktionen mit den Stationssystemen. Auf diese Weise kann dieselbe grundlegende Terminalstruktur für unterschiedliche Spielabschnitte genutzt werden.

Die Auswahl einer Terminalfunktion wird intern über die Variable `current_method()` verwaltet. Nach Bestätigung der Auswahl wird die ausgewählte Terminalaktion einer entsprechenden ausführenden Funktion zugeordnet. Missionsrelevante Aktionen werden gleichzeitig an den in Abschnitt 5.2.1 beschriebenen Missionsmanager übermittelt. Listing 5.6 zeigt exemplarisch die Zuordnung ausgewählter Terminalfunktionen.

LISTING 5.6: Weiterleitung ausgewählter Terminalaktionen

```
func _on_method_pressed():
    match current_method:
        "request_access":
            MissionManager.complete_subtask(
                "security_access",
                "Request clearance in terminal"
            )
            execute_request_access()
        "send_repair_bot":
            MissionManager.complete_subtask(
```



```

        "repair",
        "Select & send repair robot"
    )
    execute_send_repair_bot()
    "send_all_robots":
    MissionManager.complete_subtask(
        "ai_core",
        "Activate core method in terminal"
    )
    execute_send_all_robots()
close_terminal()

```

Die konkrete Ausführung der Aktionen erfolgt über separate Funktionen und die jeweils zuständigen Stationsobjekte. So wird beispielsweise über die Funktion `execute_request_access()` die Methode `grant_access()` der Sicherheitstür aufgerufen³. Das Terminal übernimmt somit hauptsächlich die Auswahl und Weiterleitung der Aktionen, während die eigentliche Spiellogik in den zuständigen Komponenten verbleibt.

Abbildung 5.7 zeigt exemplarisch die Benutzeroberfläche eines Terminals mit den für den jeweiligen Spielbereich verfügbaren Funktionen.

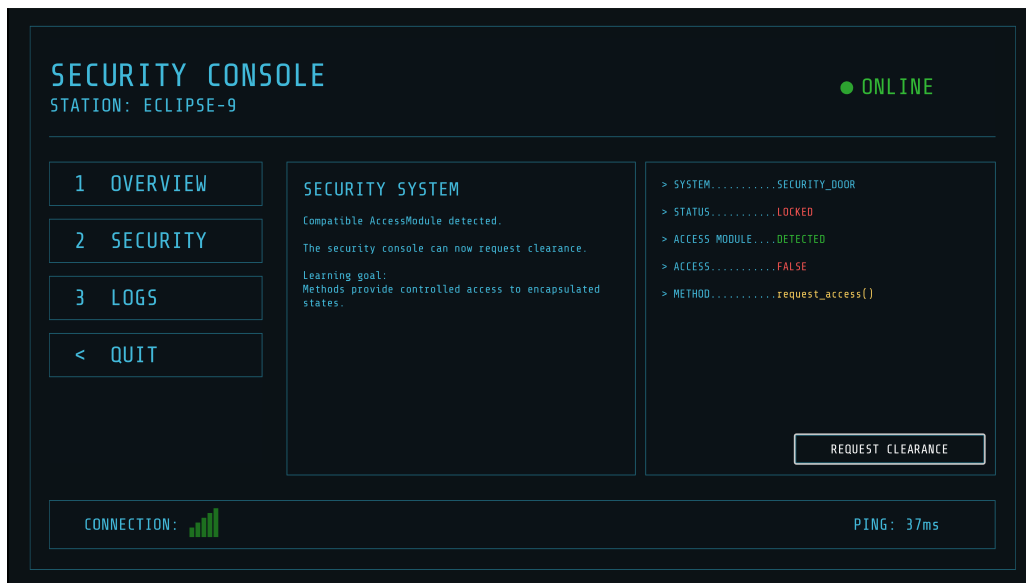


ABBILDUNG 5.7: Benutzeroberfläche eines Terminals mit Funktionen

Durch diese Trennung bleiben die Terminaloberfläche, die Stationslogik und die Missionsverwaltung weitgehend voneinander getrennt. Gleichzeitig kann die grundlegende Terminalstruktur für unterschiedliche Level wiederverwendet und um weitere Funktionen ergänzt werden. Damit wird die in Abschnitt 4.4 vorgesehene modulare und erweiterbare Softwarestruktur auch innerhalb des Terminal- und Stationssystems umgesetzt.

5.3 Benutzeroberfläche

Die Benutzeroberfläche von *DSD* setzt sich aus dem Head-up-Display (HUD), die Mini-map sowie verschiedene Menüoberflächen zusammen. Die grundlegenden Gestaltungsentscheidungen der Benutzeroberfläche wurden bereits in Abschnitt 4.4.3 beschrieben. Im Folgenden wird daher insbesondere deren technische Umsetzung und Einbindung in die zuvor beschriebenen Spielsysteme betrachtet. Die Komponenten der Benutzeroberfläche wurden innerhalb von Godot überwiegend als **CanvasLayer**- beziehungsweise **Control**-Elemente umgesetzt. Dadurch können Oberflächenelemente unabhängig von der Position der Spielfigur und der Kamerabewegung dargestellt werden. Die einzelnen Komponenten besitzen eigene Skripte und greifen bei Bedarf auf die Zustände der zugehörigen Spielsysteme zurück. Auf diese Weise bleiben Darstellung und Spiellogik weitgehend voneinander getrennt. Das HUD ist während des regulären Spielverlaufs dauerhaft sichtbar und bündelt die für den Spielenden unmittelbar relevanten Informationen. Dazu gehören insbesondere der aktuelle Missionsfortschritt, das Inventar sowie situationsabhängige Dialog- und Interaktionshinweise. Die dargestellten Informationen werden von den Spielsystemen bereitgestellt und bei Änderungen des Spielzustands aktualisiert. So führt beispielsweise der Abschluss einer Untermission im Missionssystem zu einer Aktualisierung der Missionsanzeige, während aufgenommene beziehungsweise abgelegte Objekte unmittelbar im Inventar dargestellt werden. Damit setzt das HUD einen Teil der in Abschnitt 4.3.3 vorgesehenen kontinuierlichen Rückmeldung über den Spiel- und Missionsfortschritt technisch um. Abbildung 5.8 zeigt das HUD während des regulären Spielverlaufs.



ABBILDUNG 5.8: HUD während des Spielverlaufs

Zur Unterstützung der Orientierung wurde zusätzlich eine Minimap implementiert. Diese stellt die aktuelle Position der Spielfigur sowie das Ziel der aktiven Mission innerhalb einer abstrahierten Darstellung der Raumstation dar. Das angezeigte Missionsziel wird anhand des aktuellen Zustands des in Abschnitt 5.2.1 beschriebenen Missionssystems aktualisiert. Nach dem Abschluss einer Aufgabe wird somit automatisch das Ziel der nachfolgenden Mission beziehungsweise Untermission dargestellt.

Die Minimap unterstützt darüber hinaus die Exploration der Spielwelt. Zu Beginn noch nicht erkundete Bereiche werden zunächst ausgeblendet und erst beim Erkunden der entsprechenden Bereiche sichtbar. Dadurch bildet die Minimap nicht unmittelbar die gesamte Raumstation ab, sondern erweitert sich entsprechend dem Spielfortschritt. Die schrittweise Aufdeckung greift damit die in Abschnitt 4.2.3 konzipierte Exploration der Raumstation auf und verknüpft die Orientierungshilfe mit dem individuellen Spielfortschritt. Über die Taste M kann die Minimap zusätzlich ein- und ausgeblendet werden. Abbildung 5.9 zeigt die Minimap mit der Position der Spielfigur und einem aktiven Missionsziel.



ABBILDUNG 5.9: Minimap mit Missionszielen

Neben den während des Spielverlaufs sichtbaren Elementen wurden verschiedene Menüoberflächen umgesetzt. Das In-Game-Menü (siehe Abbildung 5.10) kann über die Taste ESC aufgerufen werden und unterbricht während seiner Anzeige den regulären Spielverlauf. Über das Menü können die Spielenden das Spiel fortsetzen, zum Hauptmenü zurückkehren oder die Anwendung beenden. Die Navigation erfolgt über die Pfeiltasten und die Bestätigung über **ENTER**. Sowohl der Wechsel zwischen den Menüpunkten als auch deren Auswahl werden durch akustisches Feedback unterstützt.

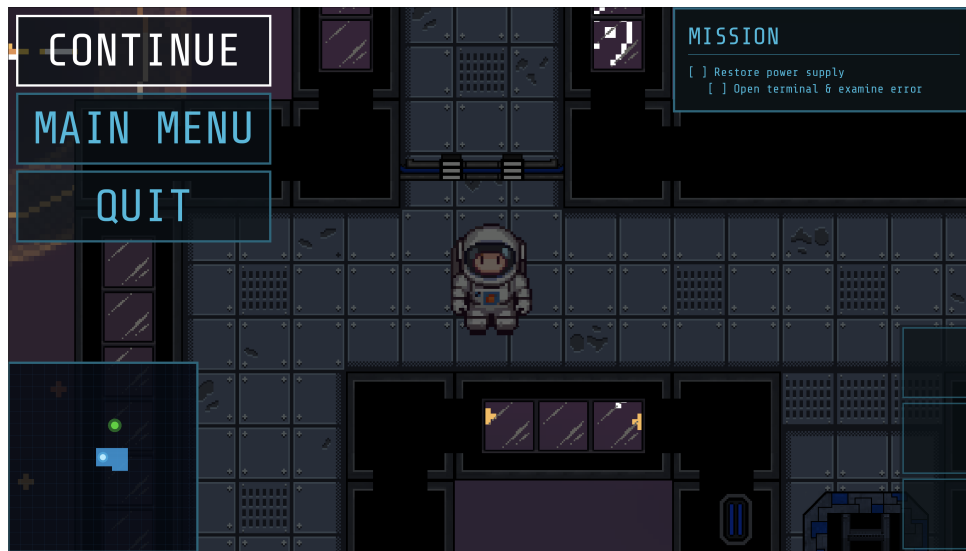


ABBILDUNG 5.10: In-Game-Menü

Das Hauptmenü (siehe Abbildung 5.11) ermöglicht den Start eines neuen Spiels, den Aufruf einer Informationsseite sowie das Beenden der Anwendung. Navigation und Bedienung entsprechen dem In-Game-Menü, sodass für beide Menüoberflächen ein einheitliches Bedienkonzept verwendet wird.

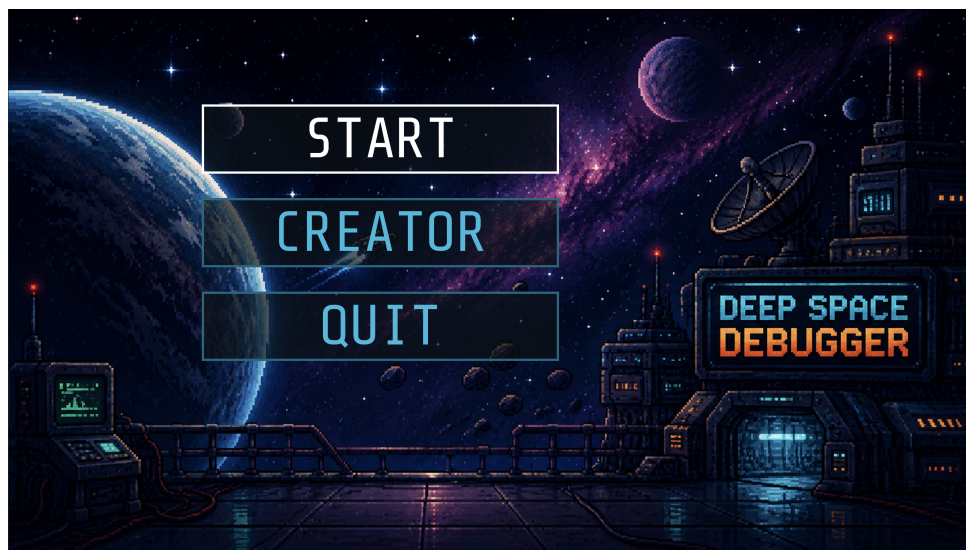


ABBILDUNG 5.11: Hauptmenü von Deep Space Debugger

Für mobile Endgeräte wurden ergänzend virtuelle Bedienelemente für Bewegung und Interaktion umgesetzt. Diese werden nur auf Geräten mit Touch-Unterstützung eingeblendet und ersetzen die entsprechenden Tastatureingaben. Die übrigen Elemente der Benutzeroberfläche bleiben davon unverändert.

6 Evaluation

In diesem Kapitel wird die Entwicklung von *Deep Space Debugger* kritisch reflektiert und hinsichtlich der in Kapitel 3 abgeleiteten Anforderungen bewertet. Hierzu werden zunächst Erfahrungen aus dem Entwicklungsprozess reflektiert und im Anschluss die Umsetzung der Anforderungen im Prototyp betrachtet. Abschließend werden die Limitationen der Arbeit dargestellt und daraus Ansatzpunkte für zukünftige Weiterentwicklungen aufgezeigt.

6.1 Reflexion der Entwicklung

Die Wahl der Godot Engine als Entwicklungsumgebung hat sich im Verlauf der Umsetzung als geeignet erwiesen. Insbesondere das Node- und Szenenkonzept unterstützte den modularen Aufbau der Spielsysteme und erleichterte deren schrittweise Erweiterung. Neue Spielmechaniken und Komponenten konnten dadurch weitgehend unabhängig von bereits bestehenden Systemen ergänzt werden.

Im Entwicklungsprozess zeigte sich, dass neben der technischen Implementierung insbesondere die konzeptionelle Verbindung der Lerninhalte mit konkreten Spielhandlungen einen wesentlichen Teil des Arbeitsaufwands ausmachte. Insbesondere die konzeptionelle Planung und die sinnvolle Verbindung der Lerninhalte mit konkreten Spielhandlungen erwiesen sich als zeitintensiv. Eine zentrale Herausforderung bestand darin, die objektorientierten Konzepte so in die Spielmechaniken einzubetten, dass diese nicht ausschließlich textuell vermittelt, sondern Bestandteil der jeweiligen Aufgaben und Interaktionen sind. Ein weiterer wesentlicher Aufwand entstand durch das Leveldesign. Sowohl die Gestaltung der Spielwelt als auch die sinnvolle Platzierung von Interaktionsobjekten, Missionszielen und Orientierungshilfen mussten sorgfältig aufeinander abgestimmt werden. Auch die Umsetzung der Benutzeroberfläche nahm mehr Entwicklungszeit in Anspruch als ursprünglich angenommen, da unterschiedliche Spielsituationen und Systemzustände berücksichtigt werden mussten. Aufgrund des begrenzten Bearbeitungszeitraums war daher eine kontinuierliche Priorisierung der geplanten Funktionen notwendig.

Für zukünftige Entwicklungsprojekte sollte insbesondere der Aufwand für das Leveldesign, die Benutzeroberfläche sowie die didaktische Einbettung der Lerninhalte bereits

während der Projektplanung großzügiger berücksichtigt werden. Die gewählte modulare Softwarestruktur bietet gleichzeitig eine geeignete Grundlage, um den entwickelten Prototyp zukünftig um weitere Spielbereiche und Mechaniken zu erweitern.

6.2 Umsetzung der Anforderungen

Im Folgenden wird bewertet, inwieweit die in Kapitel 3 zusammengeführten Anforderungen im entwickelten Prototyp umgesetzt beziehungsweise berücksichtigt wurden. Die Bewertung dient gleichzeitig der Einordnung der Forschungsfrage, inwiefern grundlegende OOP-Konzepte durch ein 2D Serious Game vermittelt und verständlich gemacht werden können. Dabei ist zwischen der technischen oder konzeptionellen Umsetzung einer Anforderung und ihrer tatsächlichen Wirkung auf die Spielenden zu unterscheiden. Da im Rahmen dieser Arbeit keine Evaluation mit der Zielgruppe durchgeführt wurde, können insbesondere hinsichtlich des Lernerfolgs, der Motivation und der Benutzerfreundlichkeit keine empirisch abgesicherten Aussagen getroffen werden.

Tabelle 6.1 stellt die zentralen Anforderungen ihrer jeweiligen Umsetzung im entwickelten Prototyp gegenüber. Die Bewertung unterscheidet zwischen umgesetzten, konzeptionell berücksichtigten und nicht überprüften Anforderungen.

Anforderung		Umsetzung im Prototyp	Bewertung
Schrittweise Vermittlung der OOP-Konzepte		Tutorial, progressive Levelstruktur, aufeinander aufbauende Missionen (siehe Abschnitte 5.2.5 und 5.2.1)	Umgesetzt
Verknüpfung von Lern- und Spielzielen		Einbettung der OOP-Konzepte in missionsbezogene Aufgaben und Interaktionen innerhalb der Spielwelt (siehe Abschnitte 5.2.3 und 5.2.6)	Umgesetzt
Unmittelbares Feedback	Feed-	Missionsanzeige, Dialoge, visuelle Hervorhebungen, akustische Rückmeldungen (siehe Abschnitte 5.2.4 und 5.2.6)	Umgesetzt
Praktische Anwendung der Lerninhalte	Anwen-	Interaktive Aufgaben, Anwendung der OOP-Konzepte innerhalb der Spielwelt (siehe Abschnitte 5.2.3 und 5.2.6)	Umgesetzt

Anforderung	Umsetzung im Prototyp	Bewertung
Verwendung fachlich korrekter Begriffe	Verwendung der OOP-Begriffe, konzeptionelle Darstellung der Lerninhalte (siehe Abschnitt 5.2.3)	Umgesetzt
Fokus auf grundlegende OOP-Konzepte	Beschränkung auf Klassen und Objekte, Kapselung, Vererbung, Objektkommunikation und Polymorphie (siehe Abschnitt 4.3.1)	Umgesetzt
Motivation durch Handlung und Progression	Narrative Einbettung, Missionsfortschritt, schrittweise Freischaltung neuer Stationsbereiche (siehe Abschnitte 5.2.1 und 5.3)	Konzeptionell berücksichtigt
Erkundbare Spielbereiche sowie wiederholbare Interaktionen mit Objekten und Stationssystemen	Erkundung der Raumstation, eigenständige Interaktion mit Objekten und Systemen (siehe Abschnitte 5.2.2 und 5.3)	Umgesetzt
Entscheidungen ohne reale Konsequenzen	Fehlerhandlungen können innerhalb der Spielwelt korrigiert und Aufgaben erneut durchgeführt werden (siehe Abschnitt 5.2.3)	Umgesetzt
Intuitive und einfache Bedienung	Einheitliches Interaktionskonzept, Tutorial, HUD, Minimap, konsistente Menüführung (siehe Abschnitte 5.2.2, 5.2.5 und 5.3)	Konzeptionell berücksichtigt
Zielgruppenorientierte Gestaltung	Ausrichtung von Lerninhalten, Steuerung und Spielgestaltung auf Studierende mit grundlegender Technikaffinität (siehe Abschnitt 5.3)	Konzeptionell berücksichtigt
Überprüfung der pädagogischen Wirksamkeit	Keine Evaluation mit der Zielgruppe im Rahmen dieser Arbeit durchgeführt	Nicht überprüft
Einfacher Zugang zum Spiel	Webexport, grundlegende Unterstützung mobiler Endgeräte (siehe Abschnitte 5.1 und 5.3)	Umgesetzt

TABELLE 6.1: Umsetzung der zusammengeführten Anforderungen im Prototyp

Die Gegenüberstellung in Tabelle 6.1 verdeutlicht, dass ein Großteil der in Kapitel 3 zusammengeführten Anforderungen im Prototyp umgesetzt werden konnte. Dies betrifft insbesondere die schrittweise Vermittlung der OOP-Konzepte, die Verknüpfung von Lern- und Spielinhalten, die praktische Anwendung der Lerninhalte sowie die vorgesehenen Feedbackmechanismen.

Im Hinblick auf die Forschungsfrage zeigt die Umsetzung, dass sich grundlegende OOP-Konzepte in konkrete Spielmechaniken, Aufgaben und Interaktionen eines 2D Serious Games übertragen lassen. Anforderungen, deren Erfüllung von der tatsächlichen Wahrnehmung oder Wirkung auf die Spielenden abhängt, können hingegen nicht abschließend bewertet werden. Dies betrifft insbesondere die Motivation, die intuitive Bedienung sowie die zielgruppenorientierte Gestaltung. Diese Aspekte wurden in der Konzeption (siehe Kapitel 4) berücksichtigt, ihre tatsächliche Wirkung müsste jedoch durch eine Evaluation mit der Zielgruppe überprüft werden. Gleiches gilt für die pädagogische Wirksamkeit, so dass auf Grundlage des entwickelten Prototyps keine Aussagen über einen tatsächlichen Lernerfolg getroffen werden können.

6.3 Limitationen

Die vorliegende Arbeit weist einige Limitationen auf, die bei der Einordnung der Ergebnisse berücksichtigt werden müssen. Eine wesentliche Limitation besteht in der fehlenden Evaluation mit der eigentlichen Zielgruppe. Aussagen über den tatsächlichen Lernerfolg, die Motivation oder die wahrgenommene Benutzerfreundlichkeit können daher nicht getroffen werden. Zudem basiert die Anforderungsanalyse auf zwei Experteninterviews, wodurch die daraus gewonnenen Erkenntnisse nur eingeschränkt verallgemeinerbar sind.

Inhaltlich konzentriert sich *DSD* auf die grundlegenden OOP-Konzepte Klassen und Objekte, Kapselung, Vererbung, Objektkommunikation und Polymorphie. Weiterführende Themen wie Entwurfsmuster und UML-Klassendiagramme wurden aufgrund des begrenzten Umfangs der Arbeit nicht berücksichtigt.

Eine weitere Limitation ergibt sich aus dem prototypischen Charakter der Umsetzung. Der begrenzte Entwicklungszeitraum schränkte insbesondere den Umfang der Spielwelt, der Spielmechaniken und der vermittelten Lerninhalte ein. Eine weiterführende Evaluation und Erweiterung des Prototyps werden daher im abschließenden Ausblick aufgegriffen.

7 Fazit und Ausblick

Ziel dieser Bachelorarbeit war die Konzeption und prototypische Entwicklung des Serious Games *Deep Space Debugger* zur Vermittlung grundlegender Konzepte der objektorientierten Programmierung. Hierzu wurden auf Grundlage der theoretischen Grundlagen, des Forschungsstands und zweier Experteninterviews Anforderungen an das Serious Game abgeleitet. Auf dieser Grundlage wurde ein didaktisches und technisches Konzept entwickelt und anschließend prototypisch umgesetzt.

Im Hinblick auf die Forschungsfrage zeigt die Arbeit, dass sich die grundlegenden OOP-Konzepte Klassen und Objekte, Kapselung, Vererbung, Objektkommunikation und Polymorphie in konkrete Spielmechaniken und interaktive Aufgaben übertragen lassen. Durch die schrittweise Einbindung in den Spielverlauf können die Konzepte spielerisch repräsentiert und mit konkreten Handlungen innerhalb der Spielwelt verknüpft werden. Die in der Anforderungsanalyse abgeleiteten Anforderungen konnten weitgehend umgesetzt oder konzeptionell berücksichtigt werden. Ob die entwickelte Vermittlungsform tatsächlich zu einem verbesserten Verständnis der OOP-Konzepte führt und damit als effektiv einzustufen ist, kann aufgrund der fehlenden Evaluation mit der Zielgruppe jedoch nicht abschließend beurteilt werden.

Für zukünftige Arbeiten stellt daher insbesondere die Evaluation des Prototyps mit Studierenden der Zielgruppe einen zentralen nächsten Schritt dar. Dabei sollten Lernerfolg, Motivation und Benutzerfreundlichkeit empirisch untersucht werden. Darüber hinaus kann *Deep Space Debugger* um weitere OOP-Konzepte, Levels, Spielmechaniken und narrative Inhalte erweitert werden. Eine Erprobung innerhalb von Lehrveranstaltungen und ein Vergleich mit klassischen Lehrmethoden oder anderen Serious Games könnten zusätzlich Aufschluss über den didaktischen Nutzen geben. Bei positiven Evaluationsergebnissen könnte der Ansatz langfristig als ergänzendes Lehrmittel zur Vermittlung der objektorientierten Programmierung eingesetzt und weiterentwickelt werden.

A Interviewleitfaden

A.1 Zielsetzung

Ziel des Experteninterviews ist es, typische Lernschwierigkeiten bei der objektorientierten Programmierung sowie didaktische Anforderungen an ein Serious Game zur Vermittlung dieser Konzepte zu ermitteln.

A.2 Struktur

Das Interview gliedert sich in die Themenbereiche Einstieg, OOP-Lehre, Lernschwierigkeiten der Studierenden, Anforderungen an *Deep Space Debugger* sowie Abschluss und Einverständniserklärung.

1. Einstieg

1. Welche Rolle nehmen Sie derzeit in der Lehre von Programmierung bzw. objektorientierter Programmierung an der HTWK Leipzig ein?
2. Welche Herausforderungen beobachten Sie bei der Vermittlung objektorientierter Programmierkonzepte besonders häufig?

2. OOP-Lehre

1. Welche grundlegenden Konzepte der objektorientierten Programmierung halten Sie für besonders wichtig?
2. Welche Lernmethoden funktionieren Ihrer Erfahrung nach besonders gut für die Vermittlung von OOP?
3. Welche Rolle spielen praktische Übungen beim Erlernen von OOP-Konzepten?

4. Bitte bewerten Sie die Bedeutung von Visualisierung und Interaktivität in der OOP-Lehre auf einer Skala von 1 bis 5

- 1 = unwichtig
- 2 = eher unwichtig
- 3 = neutral
- 4 = eher wichtig
- 5 = sehr wichtig

Bitte begründen Sie Ihre Einschätzung kurz.

3. Lernschwierigkeiten der Studierenden

1. Bei welchen OOP-Konzepten treten bei Studierenden besonders häufig Verständnisprobleme auf?
2. Welche typischen Missverständnisse treten bei den Studierenden auf?

4. Anforderungen an *Deep Space Debugger*

Kontext: Im Folgenden beziehen sich die Fragen auf ein 2D-Serious Game, das Studierende ergänzend zur OOP-Lehre nutzen sollen. Ziel ist die spielerische Vermittlung grundlegender OOP-Konzepte durch interaktive Aufgaben, Rätsel und Gameplay-Mechaniken.

1. Sehen Sie Potenzial in Serious Games zur Vermittlung objektorientierter Programmierung?
2. Welche Grenzen oder Herausforderungen sehen Sie beim Einsatz von Serious Games in der OOP-Lehre?
3. Welche OOP-Konzepte sollten in einem Serious Game unbedingt behandelt werden?
4. Welche OOP-Konzepte sind Ihrer Meinung nach zu komplex für ein Serious Game?
5. Welche didaktischen, spielerischen oder technischen Anforderungen müsste ein Serious Game erfüllen, damit tatsächlich ein Lernerfolg entstehen kann?

B Experteninterviews

B.1 Transkription des ersten Experteninterviews (Experte A)

Hinweis: Das Interview wurde automatisiert transkribiert und anschließend hinsichtlich offensichtlicher Transkriptions- und Spracherkennungsfehler sprachlich bereinigt. Inhaltliche Aussagen wurden dabei nicht gekürzt oder verändert. Nicht eindeutig rekonstruierbare Passagen sind entsprechend gekennzeichnet.

I = Interviewer

E = Experte

B.1.1 Rolle in der Programmierlehre

I: Zunächst grundlegend zum Hintergrund: Im Rahmen meiner Bachelorarbeit entwickle ich ein Serious Game zur Vermittlung der objektorientierten Programmierung. Ziel des Interviews ist es, typische Lernschwierigkeiten bei der Vermittlung von OOP festzustellen und herauszufinden, welche didaktischen Anforderungen Sie an ein Serious Game stellen würden, wenn Sie dieses unterstützend in Ihrer Lehre einsetzen würden.

Meine erste Frage wäre zunächst: Welche Rolle nehmen Sie im Rahmen der HTWK bei der Lehre von Programmierung und insbesondere objektorientierter Programmierung ein?

E: Die Lehre, die ich gerade betreibe, beschäftigt sich mit objektorientierter Programmierung. Ungefähr zwei Drittel sind objektorientierte Programmierung. Das erste Drittel ist eher eine Wiederholung des ersten Semesters, also imperative Konstrukte.

B.1.2 Herausforderungen bei der Vermittlung

I: Welche grundlegenden Herausforderungen haben Sie bisher bei der Vermittlung der jeweiligen Konzepte der Objektorientierung festgestellt?

E: Bis vor zwei Jahren lagen die Verständnisschwierigkeiten eher auf einer höheren Ebene, beispielsweise bei der Interaktion von Objekten miteinander. Seit circa zwei Jahren mache ich die Beobachtung, dass die Studierenden generell handwerkliche Schwierigkeiten bei der Programmierung haben. Die Probleme verlagern sich also viel weiter nach unten. Man kommt teilweise gar nicht mehr dazu, die eigentlichen Inhalte zu behandeln, weil es schon an ganz einfachen Sachen scheitert.

B.1.3 Wichtige OOP-Konzepte

I: Welche grundlegenden Konzepte, beispielsweise Klassen und Objekte, halten Sie für besonders wichtig für den Einstieg in die objektorientierte Programmierung?

E: Wichtig sind Klassen und der Unterschied zwischen Klasse und Objekt. Dann die Unterscheidung: Was passiert eigentlich zur Compile-Zeit und was zur Laufzeit? Das ist wichtig zu verstehen. Außerdem Objekt- beziehungsweise Klassenhierarchien und Konstrukturen und dann einfache Patterns. Also Objekterzeugung und Objektinteraktion.

I: Also auch die Kommunikation zwischen Objekten?

E: Genau.

B.1.4 Geeignete Lehr- und Lernmethoden

I: Haben sich für Sie Lehrmethoden herauskristallisiert, mit denen sich diese Inhalte besonders gut vermitteln lassen?

E: Am Anfang steht man bei der objektorientierten Programmierung vor dem Problem: Das ist ein neues Ding. Man muss viel mehr schreiben als vorher. Die didaktische Herausforderung dabei ist, dass die objektorientierte Programmierung ihre Vorteile eigentlich erst ausspielen kann, wenn die Projekte größer werden.

Das heißt, man müsste eigentlich mit einem großen Projekt anfangen und dort schauen: Wie kann ich das vereinfachen? Objektorientierte Programmierung ist eigentlich objektorientierte Modellierung. Wie kann ich ein gegebenes Problem auf dem Computer umsetzen, ohne dass ich große Abstraktionen machen muss?

Wenn ich beispielsweise meine ganzen Spieler auf Arrays abbilden würde, hätte ich mehrere Abstraktionsebenen, die ich gar nicht brauche. Bei der Modellierung sage ich “Spieler” und dann habe ich ein Objekt “Spieler”. Das bildet diesen Spieler aus der realen Welt direkt als Spieler in der virtuellen Welt ab. Ich muss also gedanklich nicht noch einmal um die Ecke denken.

Das ist der große Vorteil. Aber das zu vermitteln, ist bei kleinen Spielzeugprojekten relativ schwierig. Das ist das Erste.

Das Zweite ist: Wenn man so weit ist und die Grundbegriffe wie Konstruktoren, Interfaces, abstrakte Klassen und so weiter geklärt hat und weiß, wofür man sie braucht, besteht die Schwierigkeit in der Interaktion miteinander. Auch das kann man zunächst nur in kleineren, überschaubaren Projekten vermitteln. Beispielsweise: Was passiert, wenn ich eine Klassenhierarchie habe und ein Objekt einer darunterliegenden Klasse erzeuge? Welche Konstruktoren werden durchlaufen?

Das muss man selbst ausprobiert haben. Gerade dieses eigene Ausprobieren und Schreiben von Programmen ist heutzutage schwierig. Die Studierenden lassen sich Programme teilweise generieren, freuen sich, wenn das richtige Ergebnis herauskommt, und meinen dann, sie hätten es selbst schreiben können. Aber das ist nicht so.

I: Sie sehen praktische Übungen also unabhängig vom jeweiligen Konzept als relevant an?

E: Ja. Egal bei welchem Konzept. Man muss es selbst tun. Das ist ein Handwerk.

B.1.5 Visualisierung und Interaktivität

I: Hinsichtlich der praktischen Übungen: Auf einer Skala von null bis fünf, wobei null “eher unwichtig” und fünf “sehr wichtig” bedeutet, wie würden Sie die Bedeutung von Visualisierung und Interaktivität bei der Vermittlung der Konzepte einschätzen?

E: Drei. Nicht ganz so wichtig, wenn es im abstrakten Sourcecode sichtbar wäre. Aber man kann das natürlich auch visuell darstellen. Ich wüsste nur noch nicht genau, wie.

B.1.6 Lernschwierigkeiten und typische Missverständnisse

I: Hinsichtlich der konkreten Lernschwierigkeiten der Studierenden: Sehen Sie Verständnisprobleme vor allem bei bestimmten OOP-Konzepten?

E: Das Verständnis leidet darunter, dass die Grundlagen nicht sitzen. Wenn das Verständnis aus dem ersten Semester nicht da ist, also beispielsweise: Wann wird was im Programm gemacht? Was passiert zur Laufzeit und was passiert zur Compile-Zeit? Dann wird es schwer, diese objektorientierten Konstrukte zu vermitteln. Das baut aufeinander auf.

I: Gibt es typische Missverständnisse bei den Studierenden, beispielsweise bei der Unterscheidung von Klassen, Objekten und Instanzen?

E: Ja, zum Beispiel: Was ist der Unterschied zwischen Klasse und Objekt? Dann: Was schreibe ich in ein Interface? Was ist der Unterschied zu einer abstrakten Klasse? Konstruktoren aufstellen und Attribute zuweisen ist gerade auch sehr schwierig.

I: Also teilweise vermeintlich einfache Dinge?

E: Ja, vermeintlich einfache Dinge.

B.1.7 Potenzial von Serious Games

I: Dann würde ich den Bogen zum Serious Game spannen. Im Spiel sollen die Grundlagen der OOP vermittelt werden und nicht die gesamte tiefergehende Materie, die innerhalb einer Lehrveranstaltung behandelt wird. Ziel ist vielmehr, eine Grundlage für die Einführung in die verschiedenen Konzepte zu schaffen. Sehen Sie grundsätzlich Potenzial für ein Serious Game, um diese Konzepte zu vermitteln?

E: Bestimmt. Es gibt bestimmt eine Menge Studierende, die so etwas gerne machen.

B.1.8 Grenzen und Herausforderungen von Serious Games

I: Sehen Sie dabei Herausforderungen oder Grenzen? Gibt es beispielsweise Konzepte, die sich nur schwer visualisieren oder innerhalb eines Spiels umsetzen lassen?

E: Schwierig umzusetzen sind wahrscheinlich die Fachbegriffe. Man muss die richtigen Begriffe an der richtigen Stelle verwenden.

Und vielleicht das eigene Schreiben beziehungsweise die praktische Anwendung anhand direkter Beispiele, die man einsetzen müsste, um weiterzukommen. Für kleinere Aufgaben kann das funktionieren.

Bei der Visualisierung bin ich etwas überfragt, wie man das am besten macht. Man könnte Klassen beispielsweise irgendwie als Kästchen darstellen. Was man in einem Spiel vielleicht gut darstellen könnte, wäre der zeitliche Ablauf, sodass man sieht, was passiert. Was macht eigentlich ein Compiler? Oder was passiert zur Laufzeit? Diese Verbindung zum Programm könnte man vielleicht visualisieren, sodass man sieht, wie etwas durchläuft.

B.1.9 Relevante Konzepte für ein Serious Game

I: Wenn Sie drei Konzepte auswählen müssten, die unbedingt vermittelt werden sollten, welche wären das?

E: Unterschiedliche Objekte, dann die Objekterzeugung allgemein, also was dabei passiert, und Attributzuweisung beziehungsweise Wertzuweisung generell.

B.1.10 Zu komplexe Inhalte

I: Sehen Sie auch Konzepte, die für ein Serious Game, das nicht zu tief in die Materie geht, zu komplex sein könnten?

E: Die Factory Method beispielsweise. Also Patterns im Allgemeinen darzustellen, könnte komplizierter werden.

B.1.11 Anforderungen an ein Serious Game

I: Welche didaktischen, spielerischen oder technischen Anforderungen würden Sie abschließend an ein Serious Game stellen, das diese Konzepte vermitteln soll?

E: Da weiß ich zu wenig über Games. Vielleicht könnte es irgendeinen Score ausgeben, zur Selbstkontrolle. Also dass es am Ende eine Bewertung gibt: Du hast die Aufgaben gut gemacht, aber hier besteht noch Potenzial nach oben. Das wäre auf jeden Fall eine Form von Feedback.

Vielleicht wäre auch immer ein Hinweis wichtig, was man sich noch einmal im Detail anschauen sollte.

B.1.12 Zugänglichkeit

I: Wie sehen Sie die Zugänglichkeit? Es gibt beispielsweise Unterschiede zwischen einem Programm, das zunächst auf dem Rechner installiert werden muss, einer direkt im Browser aufrufbaren Anwendung oder einer mobilen Anwendung.

E: Wenn es ganz einfach sein soll, dann würde ich es im Browser aufrufbar machen. Aber geschickt wäre es schon, wenn die Studierenden es erst herunterladen und installieren müssten, weil auch das einen Lerneffekt haben könnte. Um es dann zu spielen, muss das Spiel natürlich gut genug sein, dass sie das auch wollen. Das heißt sich ein bisschen.

Es wäre schön, wenn es nicht einfach nur “Klick und fertig” wäre, sondern die Studierenden auch Lust darauf hätten, sich ein bisschen damit zu beschäftigen. Für den Anfang ist natürlich “Browser starten und los geht’s” auch okay. Das wäre auch für einen einfachen Einsatz praktisch, weil man beispielsweise einfach einen Link teilen könnte.

Auf das Handy würde ich mich nicht fokussieren. Das spielt keine so große Rolle.

I: Gut. Vielen Dank.

B.2 Gesprächsnotizen des zweiten Experteninterviews (Experte B)

Hinweis: Die interviewte Person wurde auf eigenen Wunsch anonymisiert. Für das Interview liegt keine vollständige Transkription vor. Die nachfolgende Dokumentation basiert auf den während des Gesprächs angefertigten Notizen und gibt die wesentlichen Aussagen entsprechend der Reihenfolge des Interviewleitfadens (siehe Anhang A) sinngemäß wieder.

B.2.1 Rolle in der Programmierlehre

- Lehrtätigkeit im Bereich angewandte Informatik mit Schwerpunkt Programmierung
- Lehrveranstaltungen mit Programmierbezug über mehrere Semester
- Vermittlung grundlegender Programmierkonzepte zu Beginn des Studiums
- Vermittlung objektorientierter Programmierung mit Java
- Spätere praxisorientierte Programmiermodule mit Fokus auf praktische Aufgaben und den Umgang mit Git
- Weitere Lehrtätigkeit unter anderem im Bereich Visualisierung und Human-Computer Interaction

B.2.2 Herausforderungen bei der Vermittlung

- Stark unterschiedliche Vorkenntnisse der Studierenden
- Teilweise fehlende Grundlagen aus der schulischen Bildung
- Notwendigkeit, fehlende Grundlagen während des Studiums nachzuholen
- Teilweise geringe eigenständige Beschäftigung mit Programmierung außerhalb der Lehrveranstaltungen
- Begrenzte verfügbare Lernzeit, insbesondere bei dual Studierenden
- Heterogenität der Studierenden und verfügbare Zeit als große Herausforderungen

B.2.3 Wichtige OOP-Konzepte und Schwierigkeiten

- Unterscheidung zwischen Klassen und Objekten
- Unterscheidung zwischen Methoden und Konstruktoren
- Überladen von Methoden
- Primitive Datentypen und Referenztypen

- Datenkapselung und deren Zweck
- Vererbung wird teilweise übermäßig eingesetzt
- Schwierigkeiten beim Verständnis von Polymorphie
- Teilweise unstrukturierte Verteilung von Code auf verschiedene Klassen
- Schwierigkeiten bei Interfaces und abstrakten Klassen
- Rekursion als allgemeine Herausforderung beim Programmieren

B.2.4 Geeignete Lehr- und Lernmethoden

- Vorherige Programmiererfahrung, beispielsweise aus der Schule, ist hilfreich
- Einsatz von Metaphern zur Vermittlung abstrakter Sachverhalte
- Tafelbilder zur Visualisierung
- Hohe Bedeutung umfangreicher praktischer Übungsserien
- Praktische Programmieraufgaben als wesentlicher Bestandteil der Lehre
- Constructive Alignment zwischen Lehre, Übungen und Prüfungsformen
- Praxisnahe Prüfungsformen
- Wettbewerbe beziehungsweise Contests als mögliche motivierende Lernform
- Codequalität gewinnt in fortgeschrittenen Veranstaltungen an Bedeutung

B.2.5 Bedeutung praktischer Übungen

- Praktische Übungen werden als besonders wichtig eingeschätzt
- Hoher Stellenwert eigenständiger Anwendung der Programmierkonzepte

B.2.6 Visualisierung und Interaktivität

- Bewertung der Bedeutung von Visualisierung und Interaktivität mit **4 von 5**
- Hoher Bezug der Zielgruppe zu digitalen Spielen
- Visuelle Vermittlung von Lerninhalten wird grundsätzlich als sinnvoll angesehen

B.2.7 Typische Missverständnisse

- Verwechslung beziehungsweise Vermischung von Klassen und Objekten
- Schwierigkeiten bei grundlegenden Programmierkonzepten
- Beispielsweise Unklarheiten über den Unterschied zwischen `return` und einer Konsolenausgabe

- Teilweise fehlerhafte Verwendung von Sprach- und Programmkonstrukten
- Missverständnisse entstehen teilweise durch eine zu geringe praktische Beschäftigung mit Programmierung

B.2.8 Potenzial von Serious Games

- Serious Games grundsätzlich sinnvoll zur Unterstützung der Lehre
- Direkte Programmierpraxis sollte durch das Spiel nicht ersetzt werden
- Einführung und Verwendung fachlicher Begriffe innerhalb des Spiels sinnvoll
- Visuelle Lernformen bieten Potenzial
- Verbindung zwischen den Inhalten des Spiels und der anschließenden Lehre wichtig
- Mögliche Aufteilung des Spiels in einzelne Level für unterschiedliche OOP-Konzepte

B.2.9 Grenzen und Herausforderungen

- Beschränkung auf einzelne beziehungsweise spezielle Lernschwierigkeiten kann problematisch sein
- Intrinsische Motivation wird gegenüber rein extrinsischen Anreizen als wichtiger eingeschätzt

B.2.10 Relevante Konzepte für ein Serious Game

- Polymorphie
- Vererbung
- Interfaces
- Komposition
- Kommunikation zwischen Objekten
- Datenkapselung
- Typsicherheit und Typinferenz
- Generics
- Rekursion

B.2.11 Schwierig spielerisch vermittelbare Inhalte

- Inhalte mit starkem Bezug zu Typen können anspruchsvoll sein

- Funktionale Programmierung sollte bei einer Fokussierung auf OOP nicht zusätzlich behandelt werden
- Sinnvolle Kapselung beziehungsweise Verteilung von Verantwortlichkeiten schwierig zu vermitteln
- Objektorientierte Analyse und objektorientiertes Design schwierig spielerisch zu integrieren
- Integration von UML- beziehungsweise Klassendiagrammen und Relationen zwischen Klassen als Herausforderung

B.2.12 Anforderungen an ein Serious Game

- Möglichst einfacher Zugang, vorzugsweise über den Webbrowser
- Einfaches Deployment
- Zuverlässige und ausreichend getestete Anwendung
- Verwendung anerkannter und fachlich korrekter Begrifflichkeiten
- Korrekte Unterscheidung beispielsweise zwischen Objekt und Instanz
- Englisch als bevorzugte Sprache im Hinblick auf die Zugänglichkeit
- Keine zu starke kompetitive Ausrichtung
- Keine notwendige Bereitstellung beziehungsweise kein Hosting durch die Lehrperson
- Anpassbare Level, beispielsweise für andere Programmiersprachen
- Multimediale Vermittlung
- Fokus auf die Grundlagen der objektorientierten Programmierung

Literaturverzeichnis

- [1] Deekshita Bundhoo and Leckraj Nagowah. Gaming with oop learn: A mobile serious game to learn object-oriented programming. pages 1–6, 2022.
- [2] Sinan Si Alhir. The object oriented paradigm. *Retrieved July, 28:2004*, 1998.
- [3] Andrew P Black. Object-oriented programming: Some history, and challenges for the next fifty years. *Information and Computation*, 231:3–20, 2013.
- [4] Irit Hadar. When intuition and logic clash: The case of the object-oriented paradigm. *Science of Computer Programming*, 78(9):1407–1426, 2013.
- [5] Peter Wegner. Concepts and paradigms of object-oriented programming. pages 7–87, 1990.
- [6] Maurizio Gabbrielli and Simone Martini. Object-oriented paradigm. In *Programming Languages: Principles and Paradigms*, pages 279–334. Springer, 2023.
- [7] Peter Van Roy. Programming paradigms for dummies: What every programmer should know. 2009.
- [8] Aimi Elliyana Rais, Shahida Sulaiman, and Sharifah Mashita Syed-Mohamad. Game-based approach and its feasibility to support the learning of object-oriented concepts and programming. In *2011 Malaysian Conference in Software Engineering*, pages 307–312. IEEE, 2011.
- [9] Nestor Lasala Jr. Edutainment: Effectiveness of game-based activities in teaching ecosystem topics. *Recoletos Multidisciplinary Research Journal*, 11, 12 2023.
- [10] Claudia Schrader. Serious games and game-based learning. In *Handbook of open, distance and digital education*, pages 1255–1268. Springer, 2023.
- [11] Alan Pitair. Aspects of game-based learning. pages 1–10, 2003.
- [12] Thomas W Malone. What makes things fun to learn? a study of intrinsically motivating computer games. pages 2–94, 1980.

- [13] Rosemary Garris, Robert Ahlers, and James E Driskell. Games, motivation, and learning: A research and practice model. *Simulation & gaming*, 33(4):441–467, 2002.
- [14] Fahima Djelil and Eric Sanchez. Game design and didactic transposition of knowledge. the case of progo, a game dedicated to learning object-oriented programming. *Education and Information Technologies*, 28(1):283–302, 2023.
- [15] Raquel Hijon-Neira, Ángel Velázquez-Iturbide, Celeste Pizarro-Romero, and Luís Carriço. Serious games for motivating into programming. In *2014 IEEE Frontiers in Education Conference (FIE) Proceedings*, pages 1–8. IEEE, 2014.
- [16] Susanne Stahringer. *Gamification und Serious Games - Grundlagen, Vorgehen und Anwendungen*. Leyh, Christian, 2017.
- [17] bildung.digital. Serious games: Spielerisch ernste inhalte vermitteln, 2026.
- [18] Clark C. Abt. Serious games. pages 1–199, 1970.
- [19] Ben Sawyer. Serious games: Improving public policy through game-based learning and simulation. 01 2002.
- [20] Elaachak Lotfi and Bouhorma Mohammed. Teaching object oriented programming concepts through a mobile serious game. *Lelaachak et al*, pages 1–6, 2018.
- [21] Yoke Seng Wong, Maizatul Yatim, and Wee Hoe Tan. Learning object-oriented programming paradigm via game-based learning game – pilot study. *The International journal of Multimedia & Its Applications*, 10:181–197, 12 2018.
- [22] Wong Yoke Seng and Maizatul Hayati Mohamad Yatim. Computer game as learning and teaching tool for object oriented programming in higher education institution. *Procedia-Social and Behavioral Sciences*, 123:215–224, 2014.
- [23] Suhni Abbasi, Hameedullah Kazi, and Kamran Khawaja. A systematic review of learning object oriented programming through serious games and programming approaches. In *2017 4th ieee international conference on engineering technologies and applied sciences (icetas)*, pages 1–6. IEEE, 2017.
- [24] Robin Hunicke, Marc Leblanc, and Robert Zubek. Mda: A formal approach to game design and game research. *AAAI Workshop - Technical Report*, 1, 01 2004.
- [25] Gede Putra Kusuma, Evan Kristia Wigati, Yesun Utomo, and Louis Khrisna Putera Suryapranata. Analysis of gamification models in education using mda framework. *Procedia Computer Science*, 135:385–392, 2018.

- [26] Rogério Junior and Frutuoso Silva. Redefining the mda framework—the pursuit of a game design ontology. *Information*, 12(10):395, 2021.
- [27] Ernesto Pacheco-Velazquez, Virginia Rodes-Paragarino, Lucia Rabago-Mayer, and Andre Bester. How to create serious games? proposal for a participatory methodology. *International Journal of Serious Games*, 10(4):55–73, 2023.
- [28] Werner Siegfried Ravyse, Anita Seugnet Blignaut, Verona Leendertz, and Alex Woolner. Success factors for serious games to enhance learning: a systematic review. *Virtual Reality*, 21(1):31–58, 2017.
- [29] Francesco Bellotti, Riccardo Berta, and Alessandro De Gloria. Designing effective serious games: opportunities and challenges for research. *International Journal of Emerging Technologies in Learning (iJET)*, 5(2010), 2010.
- [30] godot.foundation. Main features, 2026.

Abbildungsverzeichnis

2.1	Game-Based-Learning-Modell nach Garris et al. [13]	10
4.1	Core Gameplay Loop von <i>Deep Space Debugger</i>	29
5.1	Entwicklungsumgebung von <i>DSD</i> in der Godot Engine	37
5.2	Visuelle Kennzeichnung eines interaktiven Objekts	40
5.3	Darstellung eines aufgenommenen Objekts im Inventar	41
5.4	In einer Dropzone platzierter Energiekern	43
5.5	Darstellung eines Dialogs über ein Hologramm	44
5.6	Darstellung der Tutorialaufgaben innerhalb der Benutzeroberfläche	45
5.7	Benutzeroberfläche eines Terminals mit Funktionen	47
5.8	HUD während des Spielverlaufs	48
5.9	Minimap mit Missionszielen	49
5.10	In-Game-Menü	50
5.11	Hauptmenü von <i>Deep Space Debugger</i>	50

Tabellenverzeichnis

3.1	Synthese der generellen Anforderungen an Serious Games aus der Literatur und der spezifischen Anforderungen an <i>DSD</i> aus den Experteninterviews .	24
4.1	Zuordnung der fachlichen Lernziele zu den Spielbereichen	26
6.1	Umsetzung der zusammengeführten Anforderungen im Prototyp	53

Quellcodeverzeichnis

5.1	Exemplarische Datenstruktur einer Hauptmission	38
5.2	Gekürzte Aktualisierung des Missionsfortschritts	39
5.3	Erkennung des Interaktionsbereichs eines Spielobjekts	40
5.4	Exemplarische Definition der Daten eines aufnehmbaren Objekts	42
5.5	Validierung eines Objekts innerhalb einer Dropzone	42
5.6	Weiterleitung ausgewählter Terminalaktionen	46

Selbständigkeitserklärung

Ich erkläre hiermit, dass ich die vorliegende Graduierungsarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Alle den benutzten Quellen wörtlich oder sinngemäß entnommene Stellen sind als solche einzeln kenntlich gemacht.

Diese Arbeit ist bislang keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht worden.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

A handwritten signature in black ink, appearing to read 'J. Hübner', written over a horizontal line.

Unterschrift

Leipzig, 30. August 2026